

Implementatie slimme AI-agent voor IT- support, self-service en intake

Realisatiedocument

Thomas Van Sande
Student Bachelor in Applied Computer Science – Machine Learning

Inhoudsopgave

1. INLEIDING	3
2. ANALYSE	4
2.1. Overzicht	4
2.2. Large Language Model (LLM)	5
2.3. Retrieval-Augmented Generation (RAG)	7
2.4. Vector Database	10
2.5. Embedding Model	12
2.6. Backend	13
2.7. Evaluatie	15
2.8. Monitoring	16
2.9. Repository	16
3. IMPLEMENTATIE	17
3.1. Architectuur en algemene opzet	17
3.2. Ingestie	18
3.3. Retrieval-Augmented Generation (RAG)	24
3.4. Intake & Ticket	29
3.5. Backend	34
3.6. Frontend	38
3.7. Evaluatie	44
3.8. Monitoring	47
3.9. Kwaliteitswaarborging en onderhoudbaarheid	49
4. BESLUIT	52
LITERATUURLIJST	53

1. Inleiding

Dit document beschrijft de realisatie van een AI-agent voor IT-support, self-service en de intake van nieuwe ICT-behoefte. Het project werd uitgevoerd in opdracht van de ICTS-dienst van Thomas More en bouwt verder op het eerder opgestelde projectplan.

Momenteel maken gebruikers gebruik van een SharePoint Service Catalog om informatie op te zoeken over IT-gerelateerde behoeftes. Indien geen relevante informatie lijkt te bestaan, moet de gebruiker manueel een ticket aanmaken via TOPdesk. Dit proces is tijdrovend, weinig gebruiksvriendelijk en vereist dat gebruikers zelf navigeren door een uitgebreide verzameling van links en pagina's.

Om dit proces te verbeteren wordt in dit project een AI-gebaseerde oplossing ontwikkeld die deze interactie automatiseert en centraliseert. Via een chatinterface kan de gebruiker rechtstreeks een vraag stellen, waarna het systeem automatisch relevante informatie ophaalt uit verschillende kennisbronnen. Indien onvoldoende context beschikbaar is, wordt een intakeprocedure opgestart waarbij gerichte vragen worden gesteld om uiteindelijk een ticket aan te maken in TOPdesk. Op deze manier wordt het volledige proces aanzienlijk efficiënter en gebruiksvriendelijker gemaakt.

Dit realisatiedocument beschrijft de volledige uitwerking van deze oplossing. Eerst wordt een analyse gemaakt van de benodigde componenten, technologieën en architecturale keuzes. Hierbij worden verschillende tools met elkaar vergeleken en onderbouwde keuzes gemaakt. Dit gebeurt met behulp van een *weighted decsion matrix*.

Vervolgens wordt de implementatie van het systeem in detail toegelicht. Hierbij wordt ingegaan op de architectuur, de ingestie van kennisbronnen, de RAG-pipeline, de intakeflow en de ticketaanmaak op TOPdesk. Deze onderdelen worden ondersteund met relevante codefragmenten en screenshots.

Daarnaast wordt aandacht besteed aan evaluatie en monitoring. De kwaliteit van de gegenereerde antwoorden wordt geanalyseerd via testsets, een LLM-as-judge en gebruikersfeedback. Monitoring maakt het mogelijk om inzicht te verkrijgen in prestaties zoals latency en tokengebruik, en ondersteunt het optimaliseren van het systeem.

Tot slot wordt het document afgesloten met een besluit waarin de belangrijkste bevindingen worden samengevat. Ook worden mogelijke verbetering besproken en zijn er aanbevelingen met het oog op de toekomst.

2. Analyse

In dit hoofdstuk wordt een analyse gemaakt van de architectuur en de verschillende componenten die nodig zijn voor de ontwikkeling van het systeem. Het doel van deze analyse is om onderbouwde keuzes te maken met betrekking tot de gebruikte technologieën en de opbouw van de applicatie.

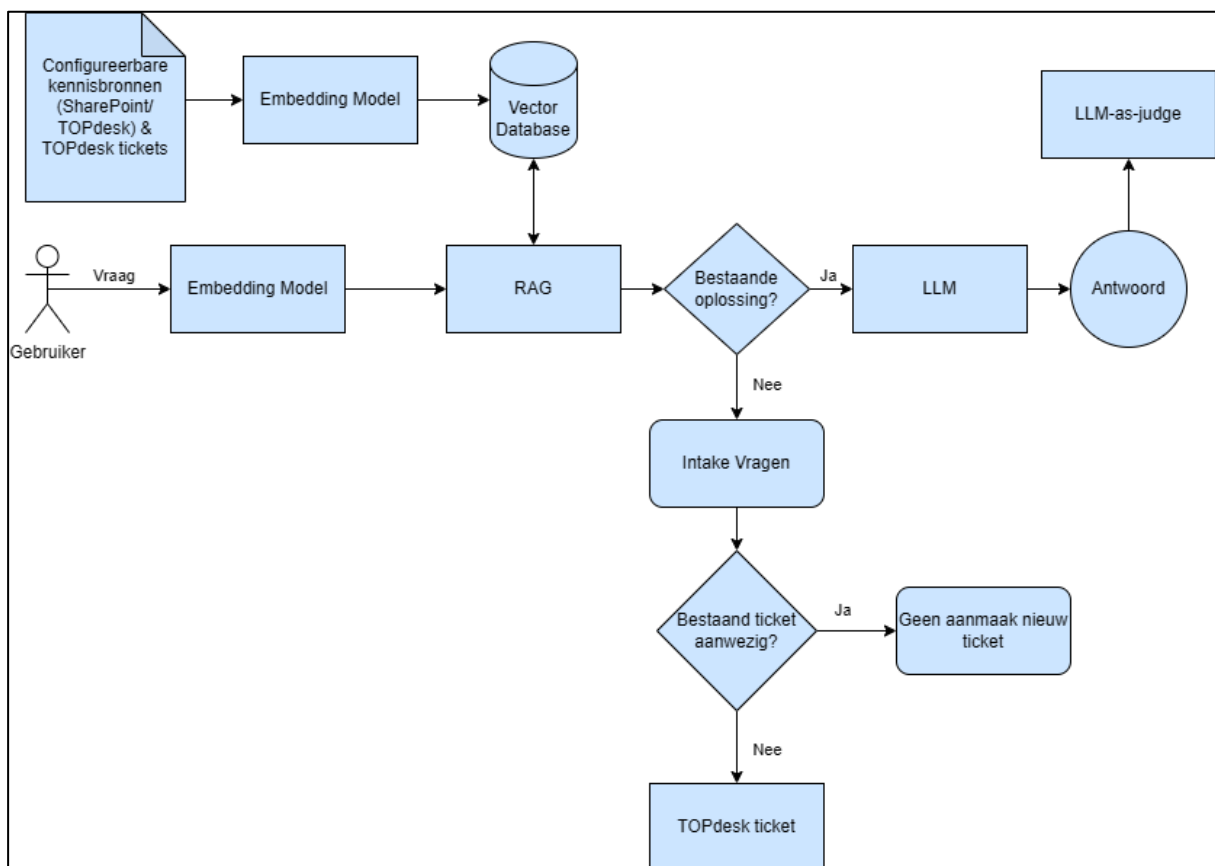
De analyse vertrekt vanuit de functionele vereisten van het systeem, waarbij gebruikers op een efficiënte manier vragen moeten kunnen stellen en, indien nodig, begeleid worden in een intakeprocedure voor het aanmaken van een ticket. Op basis hiervan wordt een architectuur uitgewerkt die deze functionaliteiten ondersteunt.

Verschiede componenten worden afzonderlijk besproken, waaronder de kennisbronnen, embeddings, vector database, retrieval, de LLM en de intakeflow. Voor elk onderdeel worden mogelijke oplossingen onderzocht en wordt gemotiveerd welke keuze het meest geschikt is binnen de context van dit project.

Deze analyse vormt de basis voor de implementatie die in het volgende hoofdstuk wordt toegelicht. De gemaakte keuzes worden daar verder uitgewerkt in een werkende oplossing.

2.1. Overzicht

Onderstaande figuur geeft een overzicht van de globale architectuur van het systeem en de verschillende componenten waaruit de applicatie is opgebouwd.



Figuur 1: Volledige flow

Het systeem vertrekt vanuit configureerbare kennisbronnen. Standaard zullen de LMS-infosite en Service Catalog op SharePoint alsook de TOPdesk kennis-items als kennisbronnen gebruikt worden. Verder kan via configuratie dit uitgebreid worden. Deze data wordt verwerkt via een embedding model en opgeslagen in een vector database, zodat semantisch zoeken mogelijk wordt.

Wanneer een gebruiker een vraag stelt, wordt deze eveneens omgezet naar een embedding. Vervolgens wordt via Retrieval-Augmented Generation (RAG) relevante context opgehaald uit de vector database. Op basis van deze context wordt bepaald of er voldoende informatie beschikbaar is om een antwoord te genereren.

Indien er voldoende relevante context aanwezig is, wordt deze samen met de oorspronkelijke vraag doorgestuurd naar een Large Language Model (LLM), dat vervolgens een antwoord genereert. Dit antwoord wordt vervolgens teruggekoppeld naar de gebruiker.

In het geval dat er onvoldoende context beschikbaar is, wordt een intakeprocedure opgestart. Tijdens deze procedure worden gerichte vragen gesteld aan de gebruiker om bijkomende informatie te verzamelen. Na afloop van de intake wordt gecontroleerd of er reeds een gelijkaardig ticket bestaat. Indien dit het geval is, wordt geen nieuw ticket aangemaakt. Anders wordt automatisch een nieuw ticket aangemaakt in TOPdesk op basis van de verzamelde context van de intakeprocedure.

Ten slotte wordt een LLM-as-judge ingezet om de kwaliteit van de gegenereerde antwoorden te evalueren. Deze evaluatie gebeurt los van de hoofdflow en heeft geen impact op de responstijd voor de gebruiker.

Om samen te vatten bestaat de AI-agent uit verschillende kerncomponenten:

- **Large Language Model (LLM)** verantwoordelijk voor het begrijpen van vragen en genereren van antwoorden
- **Retrieval-Augmented Generation (RAG):** zorgt voor het ophalen van relevante context uit externe bronnen en die mee te geven aan de LLM
- **Vector Database:** opslag en opzoeking van embeddings
- **Embedding Model:** omzetting van tekst naar vectorrepresentaties
- **Backend service:** orchestratie van de volledige flow en integratie met externe systemen
- **Evaluatiecomponent:** controle en verbetering van de kwaliteit van antwoorden

2.2. Large Language Model (LLM)

Gekozen model: Llama-3.2-3B-Instruct

Een Large Language Model (LLM) vormt een essentieel onderdeel binnen dit project, aangezien het verantwoordelijk is voor het begrijpen van gebruikersvragen en het genereren van antwoorden. Via het RAG-framework ontvangt de LLM niet alleen de gebruikersvraag, maar ook context gebaseerd op de kennisbronnen om de vraag correct en voldoende te kunnen beantwoorden.

Bij de selectie van een geschikt model moet rekening worden gehouden met verschillende criteria. Aangezien de ontwikkeling en uitvoering lokaal plaatsvinden op een laptop met beperkte hardwarecapaciteiten, is resource-efficiëntie een belangrijke factor. Daarnaast moet het model in staat zijn om Nederlandstalige vragen correct te interpreteren en kwalitatieve antwoorden te genereren. Ook compatibiliteit met RAG-systemen, gebruiksgemak en beschikbare documentatie spelen een rol in de keuze.

De belangrijkste selectiecriteria zijn:

- Ondersteuning voor de Nederlandse taal
- Compatibiliteit met RAG-frameworks
- Mogelijkheid tot lokaal gebruik
- Gebruiksgemak
- Resource-efficiëntie
- Kwaliteit van redenering en tekstgeneratie
- Beschikbaarheid van documentatie en ondersteuning
- Mogelijkheden tot fine-tuning en aanpassing
- Correctheid en consistentie van antwoorden
- Privacy
- Latency

Overwogen opties:

- Llama-3.2-3B-Instruct
- Qwen2.5-1.5B-Instruct
- Mistral-7B-Instruct-v0.3

Grotere modellen bevatten aanzienlijk meer parameters en zijn daardoor zwaarder in gebruik. Aangezien het systeem lokaal wordt uitgevoerd op hardware met beperkte capaciteit, wordt gebruikgemaakt van *quantization*. Hierbij worden de modelgewichten omgezet naar een compacter formaat (bijvoorbeeld van FP32 naar int8), waardoor het geheugengebruik en de rekenkost afnemen. Dit gaat gepaard met een beperkte daling van de nauwkeurigheid, maar laat toe om krachtigere modellen toch lokaal te gebruiken.

Criterium	Gewicht	Llama-3.2	Qwen2.5	Mistral
Nederlands begrijpen	5	4	4	4
RAG prestatie	5	4.5	4	4.5
Lokaal gebruik	5	4.5	5	3.5
Makkelijk te gebruiken	3	4	4	4
Redeneren / chatten / begrijpen	4	4.5	4	5
Resource-efficiëntie	4	4	5	3
Online ondersteuning	3	5	4	4
Fine-tuning / customization	4	4	4	4
Correctheid & consistentie	5	4.5	4	4.5
Privacy	4	4	2	4
Latency	4	4	5	3
Totale score		196.5	189	182.5

Tabel 1: Weighted Decision Matrix voor Large Language Model (LLM)

Llama-3.2 is een open-source model ontwikkeld door Meta en is instruction-tuned, wat betekent dat het geoptimaliseerd is voor het interpreteren van instructies en het genereren van consistente antwoorden. Het model ondersteunt meerdere talen, waaronder Nederlands, en is goed inzetbaar binnen een RAG-context. Daarnaast zijn er uitgebreide documentatie en ondersteuning beschikbaar, wat de implementatie vergemakkelijkt.

Het Qwen model vormt een efficiënt alternatief dankzij het lagere aantal parameters, wat resulteert in snellere uitvoering en lagere resource belasting. Hoewel het model goede prestaties levert, wordt rekening gehouden met mogelijke privacyoverwegingen gezien de Chinese herkomst van het model. Hierdoor is het moeilijk in te schatten wat met de data van de kennisbronnen gebeurt bij het oproepen van de LLM.

Mistral-7B is een open-source LLM ontwikkeld door Mistral AI. Het voordeel met dit model is dat het ontwikkeld is door een Frans bedrijf. Dit maakt het interessant in het kader van naleving van privacyregels conform de EU-regelgeving. Het is het zwaarste van de drie modellen en ook al zal quantization toegepast worden, zal dit model nog altijd het meeste vergen wat betreft resources en rekenkracht. Desondanks zal de kwaliteit het best zijn. Het is ook een Europees model getraind op meerdere talen, wat betekent dat Nederlands begrip en generatie ook geen probleem mogen zijn. Er is uitgebreide integratie met RAG en online is voor dit model ook veel informatie en documentatie te vinden.

Daarom is beslist om te kiezen voor het Llama-3.2 model. Door quantization toe te passen kan dit model gebruikt worden op een laptop met beperkte resources. Dit model is tevens sterk getraind op het begrijpen van diverse talen en is een populaire keuze. Er is veel informatie online beschikbaar en het kan makkelijk geïntegreerd worden in de oplossing. Het is handig dat dit model ook al getraind is met instruction tuning zodat het beter is om met diverse vragen te kunnen omgaan. Met het Qwen model is er wel een sterk alternatief mocht dit nodig zijn. Dan moet alleen de afweging gemaakt worden omtrent de origine uit China. Mistral kan mogelijks een optie zijn mits quantization wordt toegepast of indien getest kan worden in een omgeving met meer resources. Tijdens dit project zal de er de mogelijkheid zijn om te testen met een machine met zwaardere capaciteit. Indien op deze machine getest wordt, zal het Mistral model hiervoor gebruikt worden.

2.3. Retrieval-Augmented Generation (RAG)

Gekozen framework: LlamaIndex

Een LLM is op zichzelf onvoldoende om consistente en betrouwbare antwoorden te genereren binnen dit project. Het antwoord moet namelijk gebaseerd zijn op context aanwezig in de kennisbronnen. Daarom is het implementeren van een RAG-systeem essentieel. Dit systeem verrijkt de LLM met context afkomstig uit de externe kennisbronnen, waardoor de kans op hallucinaties aanzienlijk vermindert en de kwaliteit van de antwoorden verhoogt.

De gebruikte kennisbronnen bestaan onder andere uit de LMS-infosite, TOPdesk kennis-items en de Service Catalog. Zoals ook eerder vermeld kunnen deze bronnen in de toekomst uitgebreid worden en is hier ook een configuratie voor voorzien. Het RAG-systeem maakt het mogelijk om deze bronnen te doorzoeken en relevante informatie mee te geven aan de LLM.

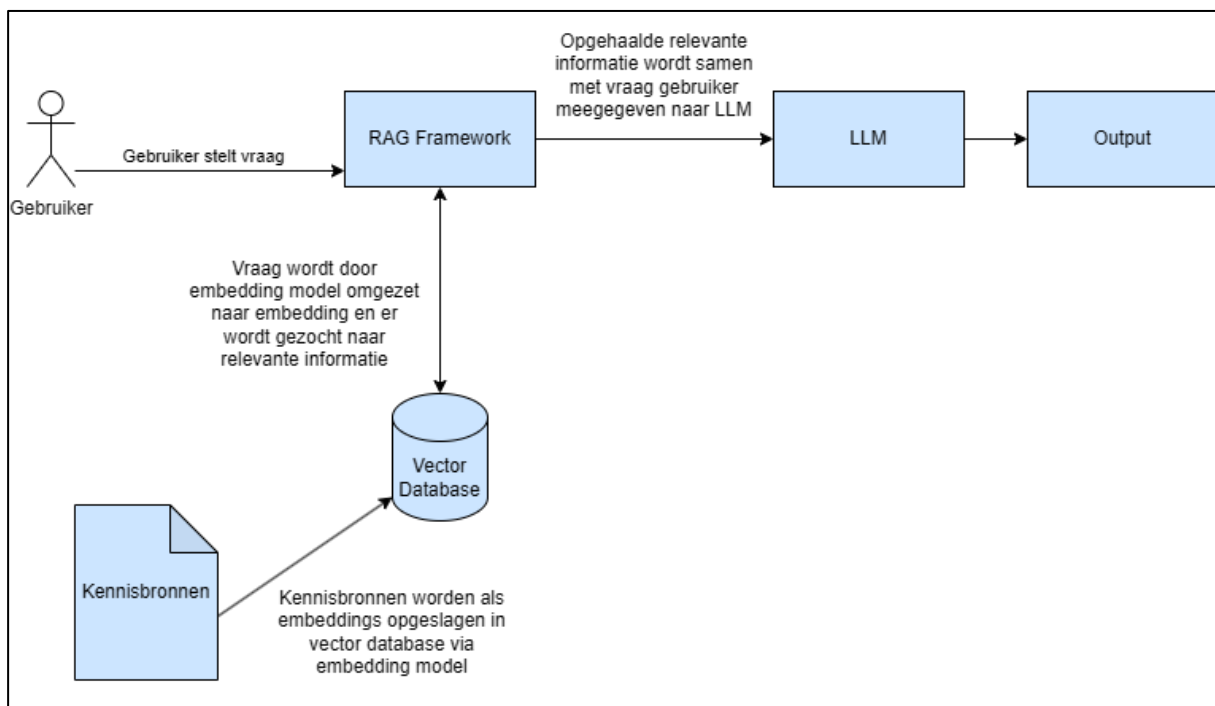
Het RAG-proces verloopt in verschillende fases. Initieel is er de indexeringsfase. In deze fase wordt alle informatie uit de kennisbronnen opgehaald en voorbereid voor efficiënte zoekopdrachten. Aangezien deze informatie vaak uit grote tekstblokken bestaat, wordt tijdens de initiële ingestie van de kennisbronnen deze eerst opgesplitst in kleinere segmenten, zogenaamde *chunks*. Dit proces wordt chunking genoemd. Hierbij wordt vaak gewerkt met een maximale chunkgrootte en eventuele overlap tussen opeenvolgende chunks om contextverlies te beperken.

Na het opsplitsen worden de chunks omgezet naar embeddings met behulp van een embedding model. Deze embeddings worden vervolgens opgeslagen in een vector database. Een embedding is een vectorrepresentatie van de chunk. Deze representatie is een numerieke voorstelling van een tekst (chunk) die semantische betekenis bevat, waardoor gelijkaardige inhoud op basis van betekenis kan worden teruggevonden.

Vervolgens is er de retrieval. Hier wordt een gebruikersvraag op dezelfde manier omgezet naar een embedding. Deze wordt dan vergeleken met de embeddings al aanwezig in de vector database via een semantische zoekopdracht. Hierbij worden de meest relevante chunks geselecteerd op basis van een *top-k* parameter. Die parameter bepaalt dus hoeveel chunks moeten worden opgehaald. De overeenkomst tussen vraag en tekstfragmenten wordt hierbij bepaald op basis van een similarity score, zoals de cosine similarity. Deze score vormt de basis voor de selectie van relevante informatie.

De geselecteerde chunks worden vervolgens samengevoegd tot één context die samen met de originele vraag naar het LLM wordt gestuurd. Dankzij deze extra context kan het model een correct en onderbouwd antwoord genereren.

In onderstaande figuur wordt een visuele flowchart weergegeven van de RAG-pipeline:



Figuur 2: RAG flow

Bij het selecteren van een geschikt RAG-framework wordt rekening gehouden met de volgende criteria:

- Integratie met LLM-modellen
- Gebruiksvriendelijkheid en eenvoud van opzet
- Ondersteuning van meerdere databronnen
- Kwaliteit van retrieval
- Resource-efficiëntie
- Beschikbaarheid van documentatie en ondersteuning
- Flexibiliteit en schaalbaarheid
- Ondersteuning vector databases
- Mogelijkheid tot het bijhouden van eerdere interacties

Overwogen opties:

- LlamaIndex
- LangChain

Criterium	Gewicht	LlamaIndex	LangChain
Integratie LLM	5	5	4
Makkelijk te gebruiken	4	5	3
Ondersteuning databronnen	5	4.5	5
Ophalen informatie	5	5	4
Resource-efficiëntie	4	5	4
Online ondersteuning	4	4	4
Flexibiliteit / schaalbaarheid	3	3	5
Ondersteuning vector database	4	5	5
Geheugen eerdere interacties	5	5	5
Totale score		182.5	169

Tabel 2: Weighted Decision Matrix voor Retrieval-Augmented Generation (RAG)

LlamaIndex is een open-source orkestratieframework gericht op integratie met Large Language Modellen. Het is gespecialiseerd in het ophalen van informatie en dit te verwerken, wat het een uiterst interessante optie maakt voor dit project.

Het framework biedt sterke ondersteuning voor vector databases, databronnen en integratie met LLM's. Daarnaast is het relatief eenvoudig in gebruik en vereist het minder complexe configuratie dan alternatieven. Ook op het vlak van resourcegebruik is het efficiënt, wat belangrijk is gezien de beperkingen van de beschikbare hardware.

LangChain is eveneens een krachtig en uitgebreid framework met veel mogelijkheden, waaronder complexere orkestratie zoals multi-step flows. Deze flexibiliteit maakt het echter ook complexer in gebruik. Voor dit project vallen veel van deze uitgebreide functionaliteiten buiten de scope, waardoor de extra complexiteit geen directe meerwaarde biedt.

Op basis van de criteria en de matrix wordt daarom gekozen voor LlamaIndex. De sterkte van dit framework ligt voornamelijk in het ophalen, structureren en geven van context aan het LLM, wat de kern vormt van dit project.

Reranking en verfijning van resultaten

Hoewel het ophalen van een top-k aantal relevante resultaten een sterke basis vormt, is het mogelijk dat deze methode niet altijd de meest relevante informatie selecteert. Dit komt doordat de initiële vergelijking gebeurt op basis van embeddings, waarbij bepaalde contextuele nuances verloren kunnen gaan. Soms wordt de semantische betekenis van een embedding ook vrij algemeen genomen, waardoor sommige vragen die niet relevant lijken bij bepaalde chunks toch aanzien als relevant hiervoor.

Om dit te voorkomen, wordt een bijkomende filtering toegepast na de initiële retrieval. Deze stap bestaat uit het gebruik van een reranker. Dit is een model dat specifiek getraind is om tekstfragmenten te evalueren in functie van een gegeven vraag. In tegenstelling tot embedding-gebaseerde methodes, die werken met numerieke vectorrepresentaties, vergelijkt een reranker de effectieve tekstinhoud van de opgehaalde chunks met de gebruikersvraag. Hierdoor kan een meer nauwkeurige inschatting van de relevantie worden gemaakt.

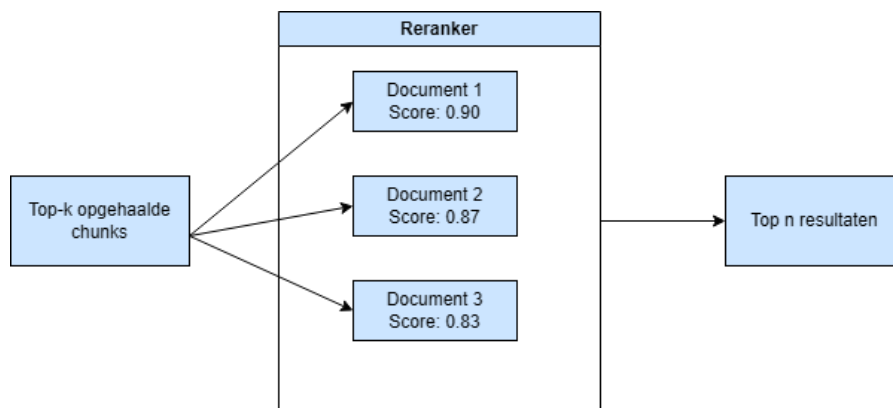
De initiële top-k resultaten worden als input gegeven aan de reranker, die deze vervolgens herschikt op basis van een nieuwe relevantiescore. Deze score houdt rekening met de volledige context van de tekst, wat leidt tot een betrouwbaardere selectie van relevante informatie.

Rerankers maken gebruik van zogenaamde *cross-encoder* architecturen, waarbij de vraag en het tekstfragment samen worden verwerkt. Dit in tegenstelling tot embedding modellen (*bi-encoders*), waarbij beide inputs afzonderlijk worden verwerkt en nadien worden vergeleken. Hoewel cross-encoders doorgaans trager zijn, leveren ze meer accurate resultaten op.

Na het herschikken van de resultaten kan een bijkomende filtering worden toegepast, bijvoorbeeld door enkel de best scorende resultaten te behouden (*top-n*) en een minimale drempelwaarde voor de relevantiescore te definiëren. Dit zorgt ervoor dat enkel de meest relevante informatie wordt doorgestuurd naar de LLM.

Voor dit project wordt gebruikgemaakt van het model *BGE-reranker-v2-m3* van BAAI, dat een goede balans biedt tussen nauwkeurigheid en prestatie. Snellere alternatieven bestaan, maar leveren doorgaans minder betrouwbare resultaten op.

Onderstaande figuur is een visueel overzicht van het reranking proces:



Figuur 3: Reranker flow

2.4. Vector Database

Gekozen database: ChromaDB

Zoals eerder vermeld moeten kennisbronnen opgeslagen worden in een vector database. Omdat in een mogelijk latere fase bestaande tickets moeten vergeleken worden met de intake, zullen ook TOPdesk tickets opgeslagen worden in de vector database. Weliswaar zullen deze terechtkomen in een andere collectie zodat de waarde van de embeddings van de kennisbronnen niet vervuild geraakt. Tijdens deze ingestiefase worden deze gegevens omgezet naar embeddings en opgeslagen in de database.

Een vector database laat toe om informatie op te slaan als vectorrepresentaties of embeddings, gegenereerd door een embedding model. Op elk moment kan deze informatie vergeleken worden met een gebruiksvraag (ook als embedding) via een *vector similarity search*. Hierbij wordt semantisch relevante informatie opgehaald die vervolgens door het RAG-systeem wordt gebruikt als context voor de LLM.

De score voor gelijkheid tussen vraag en chunks gebeurt via de *cosine similarity*. Dit is een metric die gebaseerd is op de cosinus en vaak gebruikt wordt in RAG-frameworks omdat de score bepaald wordt door gelijkheid in betekenis (richting van de vector in plaats van lengte).

Bij het selecteren van een vector database wordt rekening gehouden met volgende criteria:

- Schaalbaarheid
- Integratie met RAG-framework
- Eenvoud van setup
- Gebruiksvriendelijkheid
- Zoeksnelheid
- Resource-efficiëntie
- Compatibiliteit met embedding modellen
- Beschikbaarheid van documentatie

Overwogen opties:

- ChromaDB
- Faiss
- Qdrant

Criterium	Gewicht	ChromaDB	Faiss	Qdrant
Schaalbaarheid	2	3.5	3.5	4
RAG integratie	5	5	5	5
Setup	4	5	4	3
Gebruiksvriendelijkheid	4	5	4	3
Zoeksnelheid	4	4	5	4.5
Resource-efficiëntie	4	4	5	3
Compatibiliteit met embedding model	5	5	5	5
Online ondersteuning	3	4	4	4
Totale score		141	141	124

Tabel 3: Weighted Decision Matrix voor Vector Database

ChromaDB is een lichtgewicht wat betreft vector databases die zich onderscheidt door zijn eenvoud en gebruiksvriendelijkheid. De setup vereist weinig configuratie en de database biedt standaard ondersteuning voor persistente opslag, waardoor extra infrastructuur niet noodzakelijk is.

Daarnaast integreert ChromaDB vlot met RAG-frameworks zoals LlamaIndex en integreert het met de meest gebruikte embedding modellen, waaronder modellen van SentenceTransformers. Dit maakt het een geschikte keuze voor dit project.

Hoewel ChromaDB beperkter is op vlak van schaalbaarheid in vergelijking met alternatieven, vormt dit geen probleem binnen de huidige scope. De focus ligt voornamelijk op efficiëntie, snelle implementatie en beperkte resourcebelasting.

Faiss is een krachtige library voor het uitvoeren van similarity search, maar biedt geen volledige databasefunctionaliteit zoals persistente opslag en metadatabaseheer. Hierdoor vereist het meer manuele configuratie, wat de complexiteit verhoogt.

Qdrant daarentegen is een meer schaalbare en productiegerichte oplossing met uitgebreide functionaliteiten, maar vereist bijkomende setup, zoals het gebruik van Docker, en brengt een hogere resourcekost met zich mee.

Op basis van deze afweging wordt gekozen voor ChromaDB, aangezien het de beste balans biedt tussen eenvoud, prestatie en integratiegemak binnen dit project.

2.5. Embedding Model

Gekozen model: paraphrase-multilingual-mpnet-base-v2

Om informatie op te slaan in de vector database moet deze eerste worden omgezet naar embeddings. Dit gebeurt met behulp van een embedding model. Een embedding is een vectorrepresentatie van tekst die semantische en contextuele betekenis bevat.

Tijdens ingestie worden alle tekstfragmenten (chunks) omgezet naar embeddings en opgeslagen in de vector database. Een vraag van de gebruiker wordt eveneens omgezet naar een embedding. Dit laat het toe om later een vergelijking te doen aan de hand van een vector similarity search, zoals al eerder besproken bij de vector database en RAG.

De kwaliteit van deze embeddings heeft een directe impact op de nauwkeurigheid van de retrieval en dus op de uiteindelijke kwaliteit van de gegenereerde antwoorden.

Bij het selecteren van een embedding model wordt rekening gehouden met de volgende criteria:

- Ondersteuning van de Nederlandse taal
- Kwaliteit van de embeddings
- Resource-efficiëntie en snelheid
- Integratie met RAG-frameworks
- Beschikbaarheid van documentatie en ondersteuning

Overwogen opties:

- paraphrase-multilingual-mpnet-base-v2
- gte-Qwen2-1.5B-instruct
- bge-m3

Criterium	Gewicht	Mpnet	Bge-m3	Qwen2
Werkt met Nederlands	5	4	5	4
Embedding kwaliteit	5	3.5	5	4
Resource-efficiëntie	4	5	3	4
RAG integratie	4	5	5	5
Online ondersteuning	3	4	4	4
Totale score		97.5	94	88

Tabel 4: Weighted Decision Matrix voor Embedding Model

Paraphrase-multilingual-mpnet-base-v2 is een SentenceTransformers embedding model dat getraind is op meerdere talen, waaronder Nederlands. Dit maakt het geschikt voor het verwerken van de gebruikte kennisbronnen.

Hoewel er modellen bestaan met een hogere embeddingkwaliteit, zoals bge-m3, brengen deze een grotere belasting op resources met zich mee. Binnen de context van dit project, waarin gewerkt wordt met beperkte hardware, vormt dit een belangrijke afweging.

Gte-Qwen2-1.5B-instruct is ook al wat zwaarder wat betreft resources aangezien het model gebaseerd is op een LLM architectuur. Tevens is het breed getraind om met meerdere talen te kunnen werken waaronder Nederlands. Ook kwalitatief zijn de embeddings zeer goed. Net zoals bij de andere 2 modellen is ook hier veel online ondersteuning voor.

Op basis van deze afweging wordt gekozen voor paraphrase-multilingual-mpnet-base-v2 als embedding model. Het model biedt een goede balans tussen kwaliteit en prestatie. De embeddings zijn voldoende nauwkeurig voor de gebruikte kennisbronnen, terwijl de resourcebelasting beperkt blijft. Daarnaast integreert het model vlot met het gekozen LLamaIndex framework.

Alternatieven bieden eveneens goede prestaties, maar vereisen meer rekenkracht en zijn minder efficiënt binnen de huidige setup.

2.6. Backend

Naast de AI-component is er een backend nodig die alle onderdelen van het systeem met elkaar verbindt en de interactie met externe systemen beheert. De backend wordt opgezet als een API-gebaseerde service die fungeert als centrale orkestratie-laag binnen de applicatie.

De backend verwerkt gebruikersvragen, stuurt het RAG-proces aan, beheert intakeprocedures en regelt de integratie met externe systemen zoals TOPdesk en SharePoint. Door deze opzet kan de AI-agent eenvoudig geïntegreerd worden in verschillende interfaces, zoals een webapplicatie of SharePoint.

De backend heeft de volgende verantwoordelijkheden:

- Verwerking van gebruikersvragen en AI-orkestratie
- Beheer van kennisbronnen en ingestie
- Intake en beslissingslogica
- Integratie met externe systemen
- Configuratiebeheer

De backend ontvangt gebruikersvragen via een API-endpoint en stuurt deze door naar het RAG-systeem. Dit systeem haalt relevante informatie op uit de vector database en geeft deze samen met de originele vraag door aan de LLM.

Op basis van deze context genereert de LLM een antwoord, dat vervolgens door de backend wordt teruggestuurd naar de interface van de gebruiker. De backend bepaalt hierbij welke flow gevolgd moet worden, bijvoorbeeld het genereren van een antwoord of het opstarten van een intakeprocedure.

De backend is verantwoordelijk voor het ophalen en verwerken van kennisbronnen, zoals SharePoint-pagina's, documenten en TOPdesk kennis-items. Deze bronnen worden tijdens de ingestiefase omgezet naar embeddings en opgeslagen in de vector database.

De ingestie laag is modulair opgebouwd, waarbij per brontype een specifieke loader wordt gebruikt. Hierdoor is het eenvoudig om nieuwe databronnen toe te voegen zonder de bestaande pipeline te wijzigen. De configuratie van deze bronnen gebeurt via een JSON-structuur, waardoor bronnen dynamisch toegevoegd, aangepast of verwijderd kunnen worden.

Wanneer een nieuw type bron wordt aangemaakt zal hiervoor een nieuwe loader moeten ontwikkeld worden die dan verbonden wordt met de configuratie. Indien een nieuwe bron wordt toegevoegd van een al bestaand type bron, moet enkel de verbinding geregistreerd met de bronconfiguratie en de nodige velden meegegeven worden die het toelaten om met de externe service te kunnen verbinden.

Tijdens de ingestie wordt gewerkt met batchverwerking om het geheugengebruik te beperken en de performance te verbeteren. Tevens zal data die wordt opgehaald, ongeacht uit welke databron, opgeschoond worden en omgezet worden naar een Markdown-structuur voor een overzichtelijke en uniforme inhoud. De opschoning behandelt zaken zoals het verwijderen van onnodige HTML-tags, overbodige witruimtes en metadata van afbeeldingen verwijderen. Afbeeldingen vallen niet binnen de scope van dit project en zorgen alleen maar voor vervuiling in de embeddings.

Het ophalen van TOPdesk kennis-items en tickets zal gebeuren met de TOPdesk REST API. Voor SharePoint gebeurt dit met de Microsoft Graph API. Voor SharePoint zal ik toegang krijgen tot een eigen tenant omgeving waardoor een select aantal pagina's, sites en documenten nagebouwd kan worden en vervolgens via de API kan worden opgehaald.

Wanneer het RAG-systeem onvoldoende relevante informatie kan ophalen, wordt dit geïnterpreteerd als een mogelijke nieuwe behoefte. In dat geval start een intakeprocedure.

De beslissing om een intake te starten gebeurt op basis van twee controles. Enerzijds wordt gekeken naar de afwezigheid van voldoende relevante resultaten na reranking op basis van drempelwaardes. Anderzijds wordt de gebruikersvraag geclassificeerd door de LLM in categorieën zoals "Support", "Algemeen" of "Irrelevant. Enkel bij support-gerelateerde vragen wordt een intakeprocedure opgestart. Dit zal voorkomen dat compleet nutteloze vragen leiden tot een intakeprocedure.

Tijdens de intake worden gerichte vragen gesteld om voldoende context te verzamelen. De verzamelde informatie wordt nadien gevalideerd door deze te vergelijken met de oorspronkelijke vraag, om misbruik van het systeem tegen te gaan. Je wil immers niet dat TOPdesk tickets betekenisloos zijn.

Wanneer de intakeprocedure afgerond is, worden de verzamelde gegevens gestructureerd en voorbereid voor het aanmaken van een ticket in TOPdesk via de API. Om duplicaten te vermijden, worden de intakegegevens eerst semantisch vergeleken met bestaande tickets. Dit gebeurt door de intake-informatie om te zetten naar embeddings en te vergelijken met de embeddings van reeds bestaande tickets via vector similarity search. Indien de gelijkenis voldoende hoog is, wordt geen nieuw ticket aangemaakt.

Hieronder volgt een overzicht van de aanwezige API endpoints aanwezig voor dit project:

Endpoint	Functie	Trigger
/ask	Vragen van gebruiker verwerken	Gebruiker stuurt chatbericht
/ingest	Ophalen en verwerken van kennisbronnen	Initiële ingestie / update
/intake/start	Start intakeprocedure	Onvoldoende context + supportvraag
/intake/answer	Verwerking intake antwoorden	Gebruiker vult intake in
/sources (GET)	Ophalen bronconfiguratie	Admin raadpleegt configuratie bronnen
/sources (POST)	Aanpassen bronconfiguratie	Admin wijzigt bronnen
/feedback	Feedback op antwoorden	Gebruiker geeft evaluatie

Tabel 5: Overzicht API endpoints met bijhorende functies en triggers

De backend wordt ontwikkeld in Python, aangezien de volledige AI-toolstack compatibel is met deze programmeertaal. Voor de API-laag wordt gebruikgemaakt van FastAPI, een performant en licht framework dat geschikt is voor het bouwen van RESTful services.

Voor het testen van de applicatie wordt een eenvoudige frontend ontwikkeld met Streamlit, die toelaat om de interactie met de backend en het volledige RAG-proces visueel in kaart te brengen.

2.7. Evaluatie

Naast het ontwikkelen van de AI-agent is het essentieel om de betrouwbaarheid en bruikbaarheid van het systeem te evalueren. De evaluatie richt zich op drie kernaspecten: de kwaliteit van de retrieval, de kwaliteit van de gegenereerde antwoorden en de correctheid van de beslissingslogica omtrent intake.

De kwaliteit van de retrieval wordt geëvalueerd door het analyseren van de relevantie van de opgehaalde context na de reranking. Hierbij worden parameters zoals het aantal resultaten (top-k bij de eerste retrieval en top-n bij de reranking) en drempelwaardes voor relevantiescores aangepast en geëvalueerd.

Door deze parameters systematisch te variëren, kan bepaald worden welke configuratie leidt tot de beste resultaten wat betreft retrieval. Indien te veel irrelevante informatie wordt opgehaald, kan de drempelwaarde verhoogd worden of het aantal opgehaalde resultaten beperkt worden. Omgekeerd kan de drempelwaarde verlaagd worden indien relevante informatie verloren gaat.

Een LLM-as-judge zal toegevoegd worden om antwoorden die door een LLM gegenereerd worden te evalueren. Hierbij analyseert een tweede LLM het antwoord op basis van de oorspronkelijke vraag en de meegegeven context.

Door specifieke evaluatiecriteria mee te geven in de prompt, zoals relevantie, volledigheid en correctheid, kan het model een beknopte evaluatie en eventueel een score genereren.

Aanvullend wordt gewerkt met een handmatig samengestelde testset bestaande uit een twintigtal representatieve vragen. Voor elke vraag worden zowel de retrieval als het gegenereerde antwoord geëvalueerd.

Deze testset maakt het mogelijk om verschillende modellen, parameters en configuraties met elkaar te vergelijken en de impact op de kwaliteit van het systeem objectief te analyseren.

Een derde manier voor het evalueren van de antwoorden gebeurt aan de hand van een feedbacksysteem. In de frontend zullen gebruikers antwoorden kunnen evalueren via een eenvoudige duimpjesfunctie (positief of negatief). Deze feedback wordt opgeslagen samen met de bijbehorende vraag en het gegenereerde antwoord. Op die manier voeg je een evaluatie van het antwoord toe die zich baseert op het perspectief van de gebruiker.

Op basis van deze evaluatiemethodes voor de gegenereerde antwoorden kunnen slechte of goede antwoorden geanalyseerd worden en kunnen patronen in de prestaties van het systeem geïdentificeerd worden, wat verdere optimalisatie mogelijk maakt.

Naast retrieval en generatie wordt ook de beslissingslogica van de intake geëvalueerd. Hierbij wordt nagegaan of intakeprocedures correct worden gestart en of onnodige ticketaanmaak vermeden wordt.

Dit gebeurt door te analyseren of de combinatie van relevantiescores na retrieval en classificatie van gebruikersvragen leidt tot correcte beslissingen binnen de backend.

2.8. Monitoring

Naast evaluatie tijdens de ontwikkeling is het ook belangrijk om het gedrag van de AI-agent tijdens gebruik te monitoren. Langfuse is een ideale tool voor het observeren van het gehele proces.

Langfuse biedt de mogelijkheid om volledige flows te bekijken en in detail te analyseren. Voor elke uitgevoerde flow kan bekeken worden hoe lang deze duurde en wat de interne gegevens zijn die worden meegegeven in elke individuele stap, zoals parameterwaarden. Elke individuele stap in de flow kan uitgebreid geanalyseerd worden.

Een belangrijk onderdeel van monitoring in een AI-gerelateerd systeem is het analyseren van tokengebruik en latency. Tokengebruik geeft inzicht in het aantal tokens dat verwerkt en gegenereerd wordt door de LLM. In een RAG-systeem bestaat de input vaak uit zowel de oorspronkelijke vraag als de meegegeven context, wat kan leiden tot een relatief hoog aantal tokens. Aangezien de kost bij veel AI-modellen gebaseerd is op het aantal verwerkte tokens, is het belangrijk om hier zicht op te hebben om de operationele kost in te kunnen schatten.

Daarnaast wordt ook de latency geanalyseerd. Dit omvat de tijd die nodig is om een antwoord te genereren na een gebruikersvraag, maar ook de duur van afzonderlijke onderdelen binnen de pipeline, zoals retrieval en ingestie. Zo kan bekeken worden welke stappen de meeste verwerking en of die eventueel geoptimaliseerd moeten worden.

Monitoring maakt het mogelijk om zowel de werking als de efficiëntie van het systeem continu te verbeteren. Door inzicht te krijgen in de verschillende stappen van de pipeline kunnen gerichte optimalisaties worden doorgevoerd op het vlak van prestaties, kosten en betrouwbaarheid.

2.9. Repository

De volledige broncode van dit project is beschikbaar via GitHub:

<https://github.com/ThomasVS98/Stage>

De repository bevat de backend, RAG-pipeline, ingestiecomponenten en configuratiebestanden die in dit document worden beschreven.

3. Implementatie

In het vorige hoofdstuk werd een analyse opgemaakt van de benodigde componenten, technologieën en architectuur van de AI-agent. Op basis van deze analyse werden onderbouwde keuzes gemaakt voor de gebruikte tools en frameworks.

In dit hoofdstuk volgt de beschrijving van de effectieve implementatie van de oplossing. De focus ligt hierbij op de concrete uitwerking en de manier waarop de verschillende componenten met elkaar samenwerken. Zo kan een duidelijk inzicht worden verworven in de belangrijkste functionaliteiten, de architectuur en de technische keuzes die tijdens de ontwikkeling zijn gemaakt. Waar nodig zal deze toelichting ondersteund zijn met gerichte stukjes code en screenshots.

Eerst wordt een overzicht gegeven van de algemene architectuur en de samenhang tussen de verschillende onderdelen. Vervolgens worden de belangrijkste componenten afzonderlijk besproken volgens de logische flow van het systeem. Daarnaast wordt ook ingegaan op implementaties die werden onderzocht, maar uiteindelijk niet werden behouden.

3.1. Architectuur en algemene opzet

De architectuur van de AI-agent bestaat uit verschillende componenten die samenwerken om gebruikersvragen te verwerken en te beantwoorden. Centraal hierin staat de backend, die fungeert als een orkestratie-laag en de communicatie verzorgt tussen de frontend, de AI-componenten (RAG, LLM...) en externe systemen (SharePoint, TOPdesk). Voor een visuele weergave van de volledige flow wordt verwezen naar Figuur 1.

Om vragen correct te kunnen verwerken, moet de LLM beschikken over relevante context. Deze informatie wordt gehaald uit de beschikbare kennisbronnen. Dit gebeurt via een initiële ingestie waarbij alle bronnen aanwezig in de bronnenconfiguratie (SharePoint pagina's en TOPdesk kennis-items), alsook bestaande TOPdesk tickets, worden opgehaald. De opgehaalde data wordt opgesplitst in kleinere segmenten (chunks) en omgezet naar embeddings. Deze embeddings worden opgeslagen in de vector database en vormen de basis voor het retrievalproces.

De verwerking van een gebruikersvraag verloopt volgens een vaste flow. Er werd bewust niet gekozen voor een volledig autonome, agentic workflow waarbij het model zelf de volgende stappen bepaalt op basis van instructies. Door te werken met een vaste geprogrammeerde flow kan meer controle over het systeem behouden worden. Eerst wordt de vraag omgezet naar een embedding, waarna via het RAG-systeem relevante informatie wordt opgehaald uit de vector database. Vervolgens wordt deze informatie gefilterd en geordend met behulp van een reranker. Indien er voldoende relevante context aanwezig is, wordt deze samen met de oorspronkelijke vraag doorgestuurd naar de LLM, die een antwoord genereert.

Wanneer er onvoldoende relevante informatie beschikbaar is, wordt een intakeprocedure opgestart. Tijdens deze procedure wordt bijkomende context verzameld via gerichte vragen. Op basis van deze informatie kan vervolgens een ticket worden aangemaakt op TOPdesk. Daarbij wordt ook gecontroleerd of er reeds een gelijkaardig ticket bestaat, om duplicatie te vermijden.

De frontend biedt een gebruiksvriendelijke interface waarin gebruikers vragen kunnen stellen, antwoorden ontvangen en de intakeprocedure kunnen doorlopen. Daarnaast is er een administrator tab voorzien zodat de data ingestie en de bronnenconfiguratie ook via de UI kan beheerd worden.

Ten slotte wordt binnen de applicatie ook aandacht besteed aan evaluatie en monitoring, waarbij de kwaliteit van de gegenereerde antwoorden en de prestaties van het systeem geanalyseerd worden. Deze aspecten worden verder toegelicht in een latere sectie.

3.2. Ingestie

De ingestie pipeline is verantwoordelijk voor het verwerken van alle kennisbronnen die gebruikt worden tijdens het beantwoorden van gebruikersvragen. Tijdens dit proces worden verschillende databronnen opgehaald, verwerkt en omgezet naar een formaat dat geschikt is voor opslag in de vector database.

De pipeline bestaat uit meerdere stappen, waaronder het ophalen van data afkomstig van externe systemen zoals SharePoint en TOPdesk, het opschonen en structureren van de inhoud en het opsplitsen (chunking) van documenten in kleinere segmenten. Informatie wordt op deze manier consistent en efficiënt gebruikt binnen de RAG-pipeline.

Bronconfiguratie en dynamische loaders

De ingestie is op basis van een bronnenconfiguratie in JSON formaat. Elke bron bevat informatie zoals het type bron (Sharepoint/TOPdesk etc), configuratieparameters specifiek aan dat type bron (zoals site ID voor SharePoint) en een veld waarmee kan worden aangegeven of de bron opgenomen moet worden in de ingestie. In 'sources.json' wordt deze informatie opgeslagen:

```
[
  {
    "id": "8e3d0909-7bfe-4b45-94a9-85cf5d598c09",
    "type": "sharepoint",
    "enabled": true,
    "config": {
      "site_id": "LMS_SITE_ID",
      "label": "LMS Site",
      "include_external_links": true
    }
  },
  {
    "id": "934586c9-f1f7-42b3-8345-bdc17b91989e",
    "type": "sharepoint",
    "enabled": true,
    "config": {
      "site_id": "SERVICE_CATALOG_ID",
      "label": "IT Service Catalog",
      "include_external_links": true
    }
  },
  {
    "id": "2586992c-74bc-4cc9-80c8-f9ead403a894",
    "type": "topdesk",
    "enabled": true,
    "config": {
      "include_kb": true,
      "incident_limit": 200
    }
  }
]
```

Figuur 4: sources.json file met opgeslagen bronnen

Het type bron bepaalt de loader die gebruikt wordt om data op te halen. In de frontend is een administrator tab voorzien waar je ook daar het bronnenbeheer kan aanpassen. Voor stage doeleinden staat het veld incident limit in de TOPdesk configuratie ook al is die configuratie voor de kennis-items. Echter is het handig om tijdens ontwikkeling op een makkelijke manier te kunnen bepalen hoeveel tickets je wenst op te halen. Er wordt geen aparte configuratie voorzien voor de tickets zelf. Deze zijn niet onderdeel van de kennisbronnen maar worden uitsluitend gebruikt om een intake te vergelijken met bestaande tickets. Daardoor worden tickets ook opgeslagen in een aparte collectie in de vector database later. Dit voorkomt dat de embeddings van de kennisbronnen in de vector database vervuld worden met ticket embeddings die geen relevantie hebben voor het genereren van een antwoord.

Om die verschillende databronnen op een uniforme manier te verwerken, wordt gebruikgemaakt van een dynamisch loader-mechanisme. In plaats van per bron een vaste logica in de ingestie pipeline te voorzien, wordt er gewerkt met een centrale registry waarin loaders geregistreerd worden op basis van hun type.

Ter illustratie is hier de belangrijkste functie in 'loader_registry.py':

```
def register_loader(source_type: str, schema: dict | None = None) -> Callable:
    """
    Registreert een loader functie voor een bepaald brontype.

    Wordt gebruikt als decorator om loaders dynamisch toe te voegen
    aan de loader registry.

    Args:
        source_type (str): Type van de bron (bv. 'sharepoint', 'topdesk').
        schema (dict | None): Optioneel schema voor configuratievalidatie.

    Returns:
        Callable: Wrapper functie die de loader registreert.
    """

    def wrapper(func: Callable) -> Callable:
        LOADERS[source_type] = func
        SCHEMAS[source_type] = schema or {}
        return func

    return wrapper
```

Figuur 5: Loader registry

Elke loader die wordt aangemaakt, wordt gekoppeld aan een specifiek brontype. Bijvoorbeeld bij het maken van een SharePoint loader voeg je dit toe boven de functie:

```
@register_loader(
    "sharepoint",
    schema={
        "site_id": {"type": "string", "required": True},
        "label": {"type": "string", "required": False},
        "include_external_links": {"type": "bool", "required": False, "default": False},
    },
)
```

Figuur 6: Loader functie koppelen aan loader registry

Zo weet het systeem dat elke bron die het type *sharepoint* toegewezen krijgt, de loader functie moet gebruiken die daar toe hoort. Ook wordt een schema toegevoegd aan de loader functie zodat elk type hetzelfde schema heeft. Hier kan men ook bepalen of een bepaald veld verplicht is of niet. Op die manier kan je ook bij aanmaak van een nieuwe bron validatie toevoegen zodat er geen foute registraties mogelijk zijn. Bij het toevoegen van een nieuw bron die al een aanwezige loader heeft (zoals een nieuwe SharePoint site) moet enkel de site ID meegeven worden, maar is verder geen configuratie nodig. Echter wanneer een totaal nieuw type bron wordt toegevoegd (zoals OneDrive) zal hiervoor wel een aparte loader moeten worden aangemaakt die via de `@register_loader` toegevoegd wordt aan de registry. Vervolgens kunnen de velden gedefinieerd worden voor dat type bron die nodig zullen zijn om de informatie op te kunnen halen.

Data ophalen via loaders

Na het selecteren van de juiste loaders voor elke bron, moet je ook effectief de data ophalen en omzetten naar gestandaardiseerde Document objecten die het RAG-systeem (LlamaIndex) kan gebruiken. Elke loader heeft zijn eigen logica.

Om de SharePoint omgeving te kunnen nabootsen is toegang verleend tot een Microsoft tenant. Op deze tenant omgeving zijn 70 pagina's aangemaakt verdeeld over de Service Catalog en de LMS-infosite. De inhoud in deze pagina's is volledig gebaseerd op de inhoud van de echte SharePoint pagina's.

Voor SharePoint wordt gebruikgemaakt van de Microsoft Graph API om zowel de pagina's als bestanden op te halen per site. Aanvankelijk is geprobeerd om de ingebouwde SharePoint Reader van LlamaIndex zelf te gebruiken. Die Reader gebruikte echter een verouderde URL om naar site pagina's te zoeken en bijgevolg konden die pagina's niet gevonden worden. Een eigen reader laat het ook toe om meer controle te hebben over het opschonen en omzetten van de data. Pagina's in de SharePoint omgeving worden geparsed door de HTML-inhoud om te zetten naar een leesbaar tekstformaat. Tevens worden links aanwezig binnen een pagina ook opgeslagen. Op die manier kan je externe links tijdens de ingestie ook mee ophalen indien gewenst. Alle inhoud wordt opgeschoond door onnodige witruimtes te verwijderen. Afbeeldingen worden niet mee verwerkt. Ze kunnen handig zijn, maar enkel wanneer je een model hebt dat zowel tekst als afbeeldingen kan verwerken. Met de huidige hardware ter beschikking is dit veel te zwaar en kan dus enkel met tekst gewerkt worden. Ook metadata van elke pagina wordt bijgehouden zodat dit later kan gebruikt worden. Na de opschoning wordt alle tekst omgezet naar een gestructureerd Markdown-formaat. Zo behoudt je titels en structuur binnen een pagina en dit kan later handig zijn voor zowel het maken van embeddings alsook de LLM.

Ook bestanden aanwezig in de SharePoint omgeving worden verwerkt. De meeste bestandstypes worden verwerkt met de 'SimpleDirectoryReader' van LlamaIndex. Maar PDF en Word bestanden kunnen qua structuur soms ingewikkeld zijn. Voor deze bestandstypes is Docling gebruikt. Dit is een open-source tool die bestanden leest. Docling kan tabellen correct lezen en tekst herkennen binnen figuren. Ook heeft het een optie om automatisch de tekst om te zetten naar Markdown. Tijdens de ingestie is het parsen door Docling tamelijk belastend op het systeem. Als de ingestie klaar is blijft het geheugen dat nodig was voor Docling enkele minuten bezet. Op een laptop creëerde dit een situatie waarbij na ingestie niet onmiddellijk vragen konden gesteld worden omdat er niet genoeg RAM aanwezig was. Om dit te optimaliseren wordt document verwerking door Docling uitgevoerd in een afzonderlijk proces. Dit vermijdt dat het hoofdproces overmatig geheugen gebruikt en zo blijft de ingestie pipeline stabiel bij grotere datasets. Ook zal onmiddellijk na document verwerking het geheugen vrijkomen.

Voor TOPdesk wordt data opgehaald via de REST API. Tickets via de 'Incidents API' en kennis-items via de 'Knowledge Base API'. Alle kennis-items worden opgehaald en aanvankelijk is er een controle om te verifiëren dat deze items inhoud bevatten. Indien er inhoud aanwezig is wordt ook hier de tekst omgezet naar Markdown. Dit gebeurt ook voor tickets. Echter wordt enkel een gelimiteerd aantal tickets opgehaald omdat er een enorm aantal tickets aanwezig zijn en deze enkel dienen voor detectie op al reeds bestaande tickets. Daarom is het niet belangrijk voor de stage-opdracht om alle tickets op te halen.

Ongeacht de bron wordt dus alle data omgezet naar een uniform Markdown-formaat. LlamaIndex kan echter deze pagina's en bestanden nog niet verwerken. Eerst moeten ze omgezet worden naar een Document object.

Dit is een voorbeeld van die omzetting met de SharePoint pagina's:

```
for page in pages:
    full_text = page["content"]

    new_doc = Document(text=full_text, metadata=page["metadata"])

    new_doc.metadata["source_type"] = "sharepoint_page"
    new_doc.excluded_embed_metadata_keys = ["url", "source_id"]
    new_doc.excluded_llm_metadata_keys = ["url", "source_id"]

    yield new_doc
```

Figuur 7: Aanmaak Document-objecten voor SharePoint pagina's met generator methode

Wanneer je een Document object hebt kan je dus bijvoorbeeld bepalen welke metadata je liever niet meer in de embeddings en LLM opneemt. Dit wil je doen omdat bijvoorbeeld URL's de betekenis van een embedding alleen maar zou vervuilen. De volledige verwerking gebeurt op een sequentiële manier waarbij documenten één voor één worden opgeleverd via een generatorpatroon. Dit voorkomt dat bij grote hoeveelheden data alles tegelijk in het geheugen wordt geladen en dan verwerkt.

Wanneer alle data is geconverteerd naar Document objecten kan de opslag in de vector database beginnen.

Chunking

Na de conversie naar Document-objecten worden deze opgesplitst in kleinere segmenten. Dit proces wordt chunking genoemd. Zonder chunking moet een embedding model te grote teksten omzetten naar embeddings. Bijgevolg zal er te veel betekenis verloren gaan in deze embeddings. Daarom wordt elke Document object in chunks opgedeeld. Als je pagina's en bestanden splitst in kleinere stukken, kan relevantere context opgehaald worden tijdens het beantwoorden van de vragen.

Bij chunking zijn er heel veel methodes. In dit project werd geëxperimenteerd met meerdere technieken, waarvan de SentenceSplitter uiteindelijk werd gekozen. Deze methode probeert zinnen en paragrafen samen te houden, zodat zinnen niet opgesplitst worden over meerdere chunks.

Bij gebruik van de SentenceSplitter spelen twee parameters een belangrijke rol. De chunk grootte en de overlap. De chunk grootte bepaalt hoeveel tekst er in één chunk wordt opgenomen. Te grote chunks kunnen leiden tot minder kwalitatieve embeddings, aangezien te veel informatie samengevat moet worden in één vector. Dit kan ervoor zorgen dat minder relevante chunks worden opgehaald tijdens retrieval. Anderzijds zorgen te kleine chunks voor een groter aantal embeddings, wat een negatieve impact kan hebben op de prestatie en latency van het systeem.

Daarnaast wordt ook een bepaalde overlap gedefinieerd tussen opeenvolgende chunks. Hierbij wordt een deel van de tekst uit de vorige chunk meegenomen naar de volgende. Dit is vooral nuttig bij contextafhankelijke informatie, zoals stappenplannen, waarbij anders belangrijke informatie verloren kan gaan wanneer slechts één chunk hiervan wordt opgehaald. De andere chunk kan dan het tweede deel bevatten, maar door gebrek aan de originele context wordt deze chunk niet als relevant genoeg beschouwd tijdens de retrieval.

In dit project werd gekozen voor de volgende configuratie:

```
splitter = SentenceSplitter(chunk_size=512, chunk_overlap=200)
```

Figuur 8: Configuratie chunking methode voor kennisbronnen

Deze parameters werden bepaald op basis van experimentele evaluatie en leveren de beste balans tussen de relevantie van opgehaalde chunks en de kwaliteit van de antwoorden. Voor TOPdesk tickets wordt een grotere chunk grootte gebruikt, aangezien tickets doorgaans één specifiek probleem behandelen en het wenselijk is om deze zoveel mogelijk als één geheel te behouden.

Naast de SentenceSplitter werd ook geëxperimenteerd met de HierarchicalNodeParser. Deze methode maakt gebruik van een hiërarchische structuur waarbij kleinere “child nodes” worden opgehaald tijdens retrieval, maar nadien uitgebreid worden naar grotere “parent nodes” die meer context bevatten. Dit kan helpen om volledige pagina’s als context aan de LLM mee te geven.

Uit testen bleek echter dat deze aanpak minder geschikt was. Door de grotere hoeveelheid tekst die aan een lichte LLM wordt aangeboden, moest het aantal resultaten sterk beperkt worden. Dit kan bijgevolg opnieuw leiden tot verlies van informatie. Om deze reden is de keuze gevallen voor de SentenceSplitter als meest efficiënte en stabiele oplossing.

Embeddings en opslag in vector database

Om chunks op te slaan in een vector database moeten deze eerst omgezet worden naar embeddings. Dit zijn numerieke representaties (vectoren) van de chunks die semantische betekenis bevatten. Voor dit project is gekozen voor ChromaDB als vector database.

Bij de initialisatie van de vector database wordt een index aangemaakt waarin collecties worden gedefinieerd en het gebruikte embedding model wordt ingesteld. Dit gebeurt zowel voor de kennisbronnen als voor de TOPdesk tickets. Tickets worden bewust in een aparte collectie opgeslagen, aangezien deze geen directe inhoudelijke waarde hebben voor het genereren van antwoorden. Het toevoegen van tickets aan dezelfde collectie zou de kwaliteit van de embeddings voor kennisbronnen negatief beïnvloeden.

```
chroma_collection = chroma_client.get_or_create_collection(  
    name="docs", metadata={"hnsw:space": "cosine"}  
)  
  
vector_store = ChromaVectorStore(chroma_collection=chroma_collection)  
storage_context = StorageContext.from_defaults(vector_store=vector_store)  
  
index = VectorStoreIndex(  
    nodes=[],  
    storage_context=storage_context,  
    embed_model=get_embed_model(),  
    show_progress=True,  
)  
  
return index, chroma_collection
```

Figuur 9: Initialisatie ChromaDB vector database en collectie

De bovenstaande code toont de initialisatie van de collectie voor kennisbronnen. Hierbij wordt in de metadata ook bepaald welke similarity metric moet worden gebruikt om gelijkenis tussen vectoren te berekenen. Voor tekstuele data is *cosine similarity* hier ideaal voor. Hierbij is het de richting van de vectoren dat vergeleken wordt om semantische gelijkenis te bepalen tussen verschillende vectoren.

Daarnaast wordt het embedding model gedefinieerd dat gebruikt wordt binnen de vector database. Dit gebeurt via de functie `get_embed_model()`. Deze functie wordt zodanig opgesteld dat tijdens runtime het embedding model slechts eenmalig laadt. Elke andere keer dat dit model wordt aangeroepen is deze al in het geheugen geladen en zal er geen vertraging tot stand komen.

Onderstaande code toont het gebruikte embedding model:

```
global _embed_model
if _embed_model is None:
    device = "cuda" if torch.cuda.is_available() else "cpu"

    logger.info(f"Embedding draait op: {device}")

    _embed_model = HuggingFaceEmbedding(
        model_name="sentence-transformers/paraphrase-multilingual-mpnet-base-v2",
        normalize=True,
        device=device,
    )
return _embed_model
```

Figuur 10: Gebruikte embedding model

Het gekozen model is licht genoeg om lokaal op een laptop te draaien en ondersteunt meerdere talen, waaronder Nederlands. Tijdens de ontwikkeling werd geëxperimenteerd met verschillende modellen. Zo bleek het model 'marco-MiniLM-L12-v2' sneller en lichter, maar leverde dit merkbaar minder kwalitatieve embeddings op. Ook werd getest met modellen van JinaAI, maar deze waren moeilijker te integreren binnen de bestaande tool stack. Het gekozen mpnet-model bood uiteindelijk de beste balans.

Na de initialisatie wordt de vector database gevuld met de gecreëerde embeddings. Dit gebeurt in batches, waarbij telkens een beperkte hoeveelheid data verwerkt en opgeslagen wordt. Deze aanpak voorkomt dat alle embeddings tegelijk in het geheugen geladen worden en draagt bij aan een efficiënter gebruik van resources. Tevens verbetert dit de schaalbaarheid van de ingestie.

```
for doc in document_generator:
    if "source_id" in doc.metadata:
        doc.doc_id = doc.metadata["source_id"]

    batch.append(doc)

    if len(batch) == BATCH_SIZE:
        nodes = splitter.get_nodes_from_documents(batch)
        index.insert_nodes(nodes)
        count += len(batch)
        batch = []

    gc.collect()
```

Figuur 11: Verwerking van documenten naar vector database in batches

Voor bestanden wordt tijdens de ingestie een tijdelijke folder gebruikt waarin deze lokaal worden opgeslagen voor verwerking. Na verwerking worden deze bestanden en de bijhorende folder opnieuw verwijderd om onnodig geheugengebruik te vermijden.

Na deze stap is de ingestie pipeline volledig voltooid. De vector database bevat nu een collectie met kennisbronnen en een aparte collectie met TOPdesk tickets.

Onderstaande code toont hoe de index wordt aangemaakt en hoe het aantal verwerkte documenten wordt teruggegeven na de ingestie:

```
index, chroma_collection = create_index()

count = process_documents(index, document_generator)

logger.info("Indexering klaar")
logger.info("Totaal aantal chunks in vector store: %s", chroma_collection.count())

return count
```

Figuur 12: Aanmaken van de index en verwerken van documenten

3.3. Retrieval-Augmented Generation (RAG)

Na de ingestie van de kennisbronnen kan deze informatie gebruikt worden binnen de RAG pipeline om gebruikersvragen te beantwoorden. De RAG pipeline bestaat uit een aantal opeenvolgende stappen waarbij relevante context wordt opgehaald, verwerkt en uiteindelijk gebruikt wordt om een antwoord te genereren via de LLM.

In tegenstelling tot een volledige autonome agentic flow, wordt hier gewerkt met een vaste pipeline. Dit zorgt voor meer controle over elke stap in het proces en maakt het mogelijk om gerichte optimalisaties toe te passen, zoals reranking en filtering van resultaten. Hiermee vermijd je ook dat onvoorspelbare gedragingen zich tijdens de verwerking voordoen.

Laden van de vector index

Bij het ontvangen van een gebruikersvraag moet eerst toegang verkregen worden tot de vector database waarin de embeddings van de kennisbronnen zijn opgeslagen. Hiervoor wordt een index geladen op basis van de gedefinieerde collectie.

De index wordt niet bij elke query opnieuw opgebouwd, maar dynamisch geladen vanuit de bestaande ChromaDB opslag. Hierbij wordt, net zoals bij het laden van het embedding model, gebruikgemaakt van caching. De index wordt éénmalig geladen en nadien hergebruikt, wat de overhead van het herhaaldelijk initialiseren van de vector database bij elke query aanzienlijk vermindert. Daarnaast wordt de index pas geladen op het moment dat deze effectief nodig is. Dit volgt het principe van lazy loading. Onderstaande code toont de functie die gebruikt wordt om de index op te halen:

```
def get_index(collection_name: str) -> Optional[VectorStoreIndex]:
    """
    Haalt een index op uit de cache of laadt deze indien nodig.

    Args:
        collection_name (str): Naam van de collectie.

    Returns:
        Optional[VectorStoreIndex]: De index uit cache or nieuw geladen.
    """
    if collection_name not in cache or cache[collection_name] is None:
        cache[collection_name] = load_collection_index(collection_name)
    return cache[collection_name]
```

Figuur 13: Ophalen vector index met caching

Deze index laat het toe om de binnenkomende query vervolgens om te zetten naar een embedding met hetzelfde model dat gebruikt werd tijdens de ingestie. Op die manier blijven de embeddings consistent.

Wanneer de vector index tijdens runtime gecachet is en er een nieuwe ingestie wordt uitgevoerd, wordt de cache expliciet leeggemaakt en wordt de index opnieuw geladen. Op deze manier blijft de data in de vector database steeds up-to-date.

Retrieval

Na het laden van de vector index is het de bedoeling om de relevante chunks op te halen uit de vector database. De binnenkomende query wordt omgezet naar een embedding en vergeleken met de embeddings van de opgeslagen kennisbronnen. Op basis van deze vergelijking wordt een aantal relevante chunks (*top k*) geselecteerd. De relevantie bepaal je aan de hand van een similarity score, gebaseerd op de cosinus.

Onderstaande code toont hoe de retrieval stap wordt uitgevoerd:

```
retriever = index.as_retriever(  
    similarity_top_k=25,  
    filters=None,  
)  
  
nodes = retriever.retrieve(query)  
return nodes
```

Figuur 14: Retrieval

Zoals de code aantoont, kan bepaald worden hoeveel chunks worden opgehaald via een *top k*-parameter. In dit project werd gekozen voor een relatief hoge waarde (25 tot 40 afhankelijk van toegang tot zwaardere apparatuur), zodat initieel een brede selectie van mogelijke context wordt verzameld. Dit is een bewuste keuze, aangezien in een volgende stap nog een filtering plaatsvindt via reranking.

Het is mogelijk om na retrieval onmiddellijk context op te bouwen en deze door te geven aan de LLM. In dat geval is het echter noodzakelijk om de *top k*-waarde lager te houden, om te vermijden dat de LLM een grote hoeveelheid en deels irrelevante context ontvangt. LLM's hebben tevens een beperkte context window. Dit wil zeggen dat er een bepaald aantal tokens is dat een LLM kan verwerken. Om die reden moet er altijd rekening gehouden worden met het limiteren van de grootte van de context die men wenst mee te geven aan het model. De similarity score alleen is echter vaak onvoldoende betrouwbaar om met een lage *top k* steeds de meeste relevante resultaten te vinden.

Om deze reden voegt dit project een extra filtering stap toe, namelijk een reranker. Eerst wordt een brede selectie chunks opgehaald via retrieval, waarna een reranker gebruikt wordt om deze verder te filteren en te ordenen. Hierdoor wordt de kans vergroot dat relevante informatie behouden blijft, terwijl irrelevante context verwijderd wordt.

Reranker

Na de retrieval stap wordt een relatief groot aantal chunks opgehaald. Deze selectie bevat vaak ook minder relevante of ruisachtige resultaten. Het is bovendien mogelijk dat chunks die een hoge similarity score krijgen niet noodzakelijk de meest relevante zijn voor de vraag. Om deze reden wordt een reranker gebruikt. Deze filtert irrelevante resultaten en behoudt de meest geschikte chunks.

In tegenstelling tot retrieval, dat gebaseerd is op embeddings en similarity scores, maakt een reranker gebruik van een techniek genaamd cross-encoding. Hierbij worden de vraag en de opgehaalde chunks samen geëvalueerd, waarbij de effectieve tekstuele inhoud van beide vergeleken wordt. Hierdoor kan de reranker nauwkeuriger bepalen welke chunks het meest relevant zijn voor een specifieke vraag.

Deze aanpak heeft echter als nadeel dat de verwerking zwaarder is. Aangezien tijdens retrieval een grotere *top k* wordt gebruikt, moet de reranker elk van deze kandidaten afzonderlijk evalueren. Dit leidt tot een hogere latency, maar resulteert in een significante verbetering van de kwaliteit van de geselecteerde context.

Onderstaande code toont de initialisatie van de reranker:

```
global _reranker
if _reranker is None:
    _reranker = SentenceTransformerRerank(model="BAAI/bge-reranker-v2-m3", top_n=3)
return _reranker
```

Figuur 15: Initialisatie reranker model

Ook hier is caching en lazy loading van toepassing. Met de *top n*-parameter wordt bepaald hoeveel van de best scorende chunks behouden blijven. Elke chunk krijgt dus via de reranker een nieuwe relevantie score. Een *top n* van 3 betekent dat enkel de drie meest relevante chunks worden doorgelaten naar de volgende stap. Bij testing met de Deep Learning station wordt de parameter verhoogd naar 5 omdat een Mistral model een grotere context window heeft en dus meer tokens kan verwerken.

Naast deze selectie is er een bijkomende filter op basis van een drempelwaarde. Enkel chunks met een voldoende hoge relevantiescore blijven behouden. Dit betekent dat er in de praktijk minder dan drie resultaten kunnen overblijven, of zelfs geen enkel.

Onderstaande code toont deze filtering:

```
reranked = reranker.postprocess_nodes(
    nodes, query_bundle=QueryBundle(query_str=query)
)

reranked = [n for n in reranked if n.score is not None and n.score >= threshold]
```

Figuur 16: Oproeping reranker met extra filtering op drempelwaarde voor score

Deze drempelwaarde zorgt ervoor dat irrelevante of zwakke matches verdwijnen. Hierdoor blijft enkel de meest relevante context over voor verdere verwerking. Het is mogelijk dat na deze filtering dus geen enkele chunk overblijft. In dat geval wordt dit verder in de pipeline behandeld als een situatie zonder bruikbare context. Er wordt dan een intent detectie uitgevoerd die eventueel kan leiden tot een intakeprocedure.

Door aanvankelijk een brede selectie te maken tijdens retrieval en deze vervolgens te verfijnen via reranking, verhoog je de kans dat alle relevante chunks gevonden worden en dat vervolgens enkel de meest relevante resultaten overblijven. Met andere woorden is er dus een sterke balans tussen precision en recall. De kans is bijgevolg groot dat een LLM de juiste context ontvangt om een kwalitatief antwoord te kunnen genereren. Tijdens het testen bleek de reranker dan ook een essentieel onderdeel van het systeem, met een aanzienlijke verbetering van de resultaten.

Contextopbouw

Na de reranking blijft een beperkte groep van relevante chunks over. Deze chunks kunnen echter niet rechtstreeks naar de LLM gestuurd worden, maar moeten eerst samengevoegd worden tot één gestructureerde context.

De context die de LLM ontvangt bestaat uit een samengestelde tekst waarin meerdere relevante fragmenten gecombineerd worden. De LLM ontvangt dus niet afzonderlijke chunks, maar één volledige input waarin alle geselecteerde informatie verwerkt is.

Tijdens deze stap wordt niet enkel de tekst van elke chunk gebruikt, maar ook bijkomende metadata zoals de titel, bron en URL. Deze informatie is afkomstig van de metadata die tijdens de ingestie werd opgeslagen. Deze informatie wordt opgenomen in een gestructureerd formaat, zodat de LLM de inhoud beter kan interpreteren en verbanden kan leggen tussen verschillende fragmenten.

Onderstaande code toont hoe de context is opgebouwd:

```
context += f"""[DOCUMENT]
Bron: {source}
Titel: {title}
URL: {url}

Tekst:
{text}
[/DOCUMENT]"""
```

Figuur 17: Opbouw context met metadata

Antwoordgeneratie

Nu de context opgebouwd is, wordt deze samen met de oorspronkelijke gebruikersvraag doorgestuurd naar een Large Language Model. De LLM zal op basis hiervan een antwoord genereren. De aansturing van de LLM gebeurt via een combinatie van een system prompt en een user prompt. De system prompt bevat richtlijnen voor het gedrag van het model, terwijl de user prompt de effectieve context en vraag bevat.

Onderstaande code toont de opbouw van deze prompts:

```
def answer_system_prompt() -> str:
    """
    Geeft de system prompt voor het genereren van antwoorden.

    Returns:
        str: System prompt.
    """
    return """Je bent een IT-assistent voor de medewerkers van de Thomas More hogeschool.

    RICHTLIJNEN:
    1. Antwoord uitsluitend op basis van de onderstaande context.
    2. Gebruik enkel expliciete informatie uit de context. Verzin nooit zelf informatie als die niet in context staat.
    3. Als een specifiek detail (zoals een knopnaam of URL) niet in de tekst staat, verzin deze dan niet.
    4. Zeg alleen "ik heb niet genoeg informatie" wanneer er GEEN bruikbare informatie in de context staat om de vraag
```

Figuur 18: Een deel van de system prompt voor het antwoord

```
def answer_user_prompt(context: str, query: str) -> str:
    """
    Genereert de user prompt met context en vraag.

    Args:
        context (str): Samengestelde context.
        query (str): De gebruikersvraag.

    Returns:
        str: Prompt voor de LLM.
    """
    return f"""
    Context:
    {context}

    Vraag:
    {query}

    Antwoord:
    """
```

Figuur 19: User prompt voor antwoord

De system prompt bevat expliciete instructies om het model te dwingen enkel gebruik te maken van de aangeleverde context en geen informatie te verzinnen (hallucineren). Daarnaast wordt er opgelegd hoe het antwoord moet gestructureerd worden, bijvoorbeeld door gebruik te maken van bullet points bij meerdere stappen. Deze richtlijnen zijn essentieel om hallucinaties te vermijden en consistente antwoorden af te dwingen. Door uitvoerig te testen op vraag en antwoord kunnen bepaalde gedragingen van het model geïdentificeerd worden, waarna je de prompt verder kan optimaliseren.

De user prompt combineert de gegenereerde context met de oorspronkelijke vraag van de gebruiker. Hierdoor beschikt het model over alle noodzakelijke informatie om een correct antwoord te formuleren. Hier is het belangrijk op te merken dat de kwaliteit van het antwoord sterk afhankelijk blijft van de kwaliteit van de retrieval. Een LLM kans immers geen kwalitatief en correct antwoord genereren indien het model niet beschikt over de juiste en volledige informatie.

De effectieve generatie van het antwoord gebeurt via een LLM die lokaal wordt uitgevoerd:

```
global _llm
if _llm is None:
    _llm = Ollama(
        model="llama3.2:3b",
        request_timeout=300,
        context_window=6140,
        temperature=0,
    )
return _llm
```

Figuur 20: Initialisatie LLM

Het gebruikte model is een lokaal Ollama-model (llama3.2:3b). Ook hier is lazy loading en caching van toepassing. Het Llama model is relatief licht, wat toelaat om op een laptop met enkel CPU een antwoord te genereren. De "3b" verwijst naar het aantal miljard parameters dat gebruikt werd tijdens de training. Bij het testen op de Deep Learning station is het Mistral-7B model gebruikt.

Via de parameter `request_timeout` wordt het proces afgebroken indien dit langer dan 300 seconden duurt. Omwille van beperkte resources wordt de context window van het model beperkt, zodat het geheugengebruik binnen de capaciteiten van de laptop blijft. Tegelijk wordt deze voldoende groot gehouden zodat het model genoeg informatie kan verwerken.

Met de parameter *temperature* kan bepaald worden hoe creatief een LLM mag zijn. Door deze op nul te zetten dwing je het model om zo goed als geen woorden te verzinnen en blijft het model zich houden aan de inhoud in de meegegeven context.

De context en prompts worden vervolgens doorgestuurd naar het model via een chatinterface:

```
messages = [
    ChatMessage(role="system", content=answer_system_prompt()),
    ChatMessage(role="user", content=answer_user_prompt(context, query)),
]

response = llm.chat(messages)

return response.message.content
```

Figuur 21: Meegeleverde prompts aan LLM

De system prompt wordt doorgegeven via de “system”-rol, die het gedrag van het model bepaalt. De user prompt wordt via de “user”-rol meegegeven en bevat de context en vraag waarop het model een antwoord moet formuleren.

De volledige flow van de RAG-pipeline wordt uitgevoerd in de ‘*pipeline.py*’ file, terwijl in ‘*qa_service.py*’ deze wordt aangeroepen en uitgebreid met bijkomende functionaliteit, zoals het toevoegen van bronverwijzingen. Die verwijzingen zijn gebaseerd op de bronnen van de chunks na reranking.

Indien er geen relevante context beschikbaar is na reranking, wordt geen antwoord gegenereerd door de LLM. In dat geval wordt een mogelijk intake opgestart aan de hand van een intent detectie. Op deze manier wordt vermeden dat het model antwoorden genereert zonder voldoende onderbouwing, wat de betrouwbaarheid van het systeem aanzienlijk verhoogt.

3.4. Intake & Ticket

Wanneer de RAG-pipeline geen relevante context kan ophalen na reranking, is het niet mogelijk om een betrouwbaar antwoord te genereren via de LLM. In plaats van een onvolledig of foutief antwoord te geven, start een intakeprocedure.

Gebruikers kunnen een vraag stellen die wijst op een nieuwe behoefte. De intake heeft als doel om bijkomende informatie te verzamelen aan de hand van gerichte vragen. Op die manier wordt voldoende context opgebouwd om, indien nodig, een ticket op TOPdesk aan te maken.

Het starten van een intake gebeurt niet automatisch voor elke vraag waarvoor geen context beschikbaar is. Eerst wordt de intentie van de gebruiker geanalyseerd om te bepalen of de vraag effectief relevant is voor de ICTS dienst. Enkel wanneer dit het geval is, zal een intakeprocedure starten.

Tijdens de intake worden gerichte vragen gesteld aan de gebruiker, waarbij de antwoorden stapsgewijs worden verzameld en gevalideerd. Deze gegevens worden tijdelijk bijgehouden in een sessie, zodat de volledige interactie over meerdere stappen heen kan verlopen.

Na afloop van de intakevragen wordt gecontroleerd of de verzamelde informatie nog relevant is ten opzichte van de oorspronkelijke vraag. Indien dit het geval is, wordt eerst nagegaan of er reeds een gelijkaardig ticket bestaat in TOPdesk. Wanneer geen gelijkaardig ticket bestaat, zal een ticket automatisch aangemaakt worden op TOPdesk.

Intent detectie

Vooraleer een intakeprocedure effectief wordt opgestart, zal eerst de intent van de gebruikersvraag geanalyseerd worden. Het doel hiervan is om te bepalen of een vraag effectief ondersteuning vereist of eerder als algemeen of irrelevant wordt beschouwd.

Deze detectie gebeurt via een LLM. Dit gebeurt via een specifieke prompt:

```
Classificeer de vraag in EXACT één categorie:
- SUPPORT
- ALGEMEEN
- IRRELEVANT

REGELS:
- Geef je antwoord ALLEEN in JSON formaat.
- GEEN uitleg.
- GEEN extra tekst.
- Gebruik exact dit formaat:

{{"intent": "SUPPORT"}}

Voorbeelden:
{{"intent": "SUPPORT"}}
{{"intent": "ALGEMEEN"}}
{{"intent": "IRRELEVANT"}}

SUPPORT:
Als de gebruiker hulp nodig heeft met IT-systemen, software of diensten.
Dit omvat:
- problemen (iets werkt niet)
- hulpvragen (hoe gebruik ik iets?)
- aanvragen (toegang, tools, VPN, accounts, etc.)
```

Figuur 22: Deel van de prompt voor intent detectie

In bovenstaande code wordt een deel van de prompt getoond die de LLM ontvangt om deze classificatie uit te voeren. Via een voorbeeld (few-shot prompting) en duidelijke richtlijnen wordt bepaald wanneer een vraag tot een bepaalde categorie behoort. Daarbovenop wordt het model expliciet opgelegd om zijn antwoord in JSON-formaat te genereren. Zo kan je de output op een gestructureerde manier verwerken.

Via de functie '*detect_intent()*' wordt vervolgens de LLM aangeroepen met de meegeleverde prompt:

```
prompt = detect_intent_prompt(query)
response = llm.complete(prompt)
raw = response.text.strip()
logger.info("RAW INTENT: %s", raw)
```

Figuur 23: LLM-call voor intent detectie

Nadat de LLM een output genereert, wordt deze verwerkt zodat je de categorie uit het antwoord kan halen. Indien het model een antwoord teruggeeft in het correcte formaat, kan je de intent direct uit de output halen. In het geval van een foutieve of onvolledige output zal geprobeerd worden om de juiste categorie toch uit het antwoord te halen.

Enkel bij de intent "SUPPORT" zal een intakeprocedure opstarten. Voor vragen die als "ALGEMEEN" of "IRRELEVANT" worden beschouwd, start geen intake.

Intakeprocedure

Wanneer de intent van de gebruikersvraag "SUPPORT" is, wordt de intakeprocedure dus opgestart. Deze procedure heeft als doel om gestructureerd bijkomende informatie te verzamelen via een reeks gerichte vragen.

De intake verloopt stapsgewijs. Bij het starten van de intake wordt een nieuwe sessie aangemaakt waarin alle antwoorden van de gebruiker worden bijgehouden. Elke sessie krijgt een unieke ID, zodat de volledige intake over meerdere stappen heen kan worden opgevolgd en bijgehouden.

Onderstaande code toont de opstart van een intake sessie:

```
def start(original_question: str) -> dict[str, str]:
    """
    Start een nieuwe intake sessie.

    Initialiseert een nieuwe sessie en retourneert de eerste vraag
    uit de intake flow

    Args:
        original_question (str): De originele vraag van de gebruiker.

    Returns:
        dict[str, str]: Bevat session_id en eerste vraag van de intake.
    """
    session_id = str(uuid.uuid4())
    session_store.create(session_id)

    session_store.update(session_id, "original_question", original_question)
    _, question = INTAKE_QUESTIONS[0]
    return {"session_id": session_id, "question": question}
```

Figuur 24: Opstart intake met aanmaak session ID

De vragen die de gebruiker tijdens de intake moet beantwoorden, zijn vooraf gedefinieerd in een vaste structuur:

```
INTAKE_QUESTIONS = [
    ("beschrijving", "Beschrijf heel beknopt het probleem:"),
    ("context", "Geef context aan je probleem/aanvraag:"),
    ("doel", "Wat werkt er niet of wat wil je bereiken?:"),
]
```

Figuur 25: Intake vragen

Dit geeft meer opties om de antwoorden te valideren alsook consistent gestructureerde tickets aan te maken. Tijdens de intake worden de antwoorden van de gebruiker gevalideerd alvorens ze worden opgeslagen. Deze validatie bestaat uit een limiet op karakters voor de korte beschrijving en controleren dat velden niet leeg zijn of te weinig karakters hebben. Ongewenste HTML-tags worden ook verwijderd.

Na elke stap wordt het antwoord opgeslagen in de sessie. Dit proces wordt herhaald tot alle intakevragen beantwoord zijn. Na afronding van de intake is er eerst een controle of de verzamelde informatie nog relevant is ten opzichte van de oorspronkelijke gebruikersvraag. Dit gebeurt via een semantische vergelijking op basis van embeddings. Indien de similarity score onder een bepaalde drempelwaarde ligt, wordt het proces stopgezet en wordt geen verdere actie ondernomen. Zo verhinder je dat een gebruiker toegang krijgt tot de intakeprocedure door een relevante vraag te stellen maar nadien irrelevante antwoorden in de intake geeft om zo de TOPdesk ticketomgeving te vervuilen.

Onderstaande code geeft een overzicht van wat er gebeurt wanneer de laatste intake vraag gesteld is:

```
answer = validate_answer(key, answer)
session_store.update(session_id, key, answer)
session = session_store.get(session_id) # session data vernieuwen na elke update
if step + 1 >= len(INTAKE_QUESTIONS):
    data = session["data"]
    original_question = session["data"].get("original_question")

    if not original_question:
        logger.warning("Geen originele vraag gevonden in sessie %s", session_id)
    else:
        if not is_relevant(original_question, data):
            logger.info("Intake niet relevant voor sessie %s", session_id)
            return {
                "done": True,
                "error": "De gegeven antwoorden lijken niet overeen te komen met je oorspronkelijke vraag.",
            }
```

Figuur 26: Verwerking laatste intake vraag met controle op relevantie

Op dit moment is alle context verzameld om een ticketaanmaak in TOPdesk op te starten.

Controleren op bestaande tickets

Alvorens een ticket op TOPdesk wordt geplaatst, gebeurt er eerst een controle op bestaande tickets in de TOPdesk omgeving die gelijklopend zijn met de intake. Dit voorkomt dat duplicaten aangemaakt worden.

Die controle gebeurt op basis van de informatie die tijdens de intake werd verzameld. Deze gegevens worden samengevoegd tot één tekstuele blok bestaande uit beschrijving, context en het doel van de aanvraag. Vervolgens wordt deze inhoud vergeleken met de bestaande tickets die opgeslagen zijn in de aparte ticket collectie binnen de vector database. Die collectie werd tijdens de ingestie opgehaald. Hier wordt dezelfde aanpak gehanteerd als bij de RAG-pipeline, namelijk een combinatie van retrieval en reranking:

```
nodes = retrieve_nodes(index, query)
nodes = rerank_nodes(nodes, query, threshold=0.40)

if not nodes:
    return None

best = nodes[0]
logger.info("Beste score: %.3f", best.score)

if best.score >= SIMILARITY_THRESHOLD:
    return {"score": float(best.score), "text": best.node.get_content()}
return None
```

Figuur 27: Detectie van gelijkaardige TOPdesk tickets

Kort samengevat worden initieel gelijkaardige tickets opgehaald via retrieval. Die worden via een reranker verder gefilterd. Vervolgens blijft enkel het best scorende ticket over. Als dit ticket een hogere similarity score heeft dan de drempelwaarde, zal dit beschouwd worden als een gelijkaardig ticket. Dan wordt de aanmaak van een nieuw ticket tegengehouden. Wanneer het best scorende TOPdesk ticket onder de drempelwaarde ligt, gaat de flow verder met een ticketaanmaak op TOPdesk.

Met deze aanpak wordt vermeden dat identieke of sterk gelijkaardige aanvragen meerdere keren als nieuw ticket worden aangemaakt, wat de efficiëntie van het systeem verhoogt en de kwaliteit van de TOPdesk ticketomgeving ten goede komt.

Aanmaken TOPdesk ticket

Wanneer geen gelijkaardig ticket aanwezig is, wordt een nieuw TOPdesk ticket aangemaakt. De inhoud van dit ticket bestaat uit de informatie die verzameld is tijdens de intakeprocedure.

De effectieve aanmaak van een ticket gebeurt via een API-call naar TOPdesk, via de 'Incident' REST API. De korte beschrijving van de aanvraag wordt toegevoegd aan het "briefDescription" veld. De context en het doel worden samengevoegd in het "request" veld.

Onderstaande code toont hoe de payload wordt opgebouwd:

```
return {
    "briefDescription": (data.get("beschrijving") or "")[:80],
    "request": f"""
Context: {data.get("context")}

Doel: {data.get("doel")}
""".strip(),
    "caller": {"dynamicName": "Test user"},
}
```

Figuur 28: Opbouw TOPdesk ticket payload

Vervolgens wordt met de juiste API authenticatie het ticket aangemaakt:

```
response = requests.post(
    url,
    json=payload,
    auth=(settings.TOPDESK_USER, settings.TOPDESK_SECRET),
    headers={"Content-Type": "application/json"},
    timeout=30,
)
```

Figuur 29: Aanmaak TOPdesk ticket

Indien de instelling "TOPDESK_ENABLED" op *False* staat, wordt een mock ticket aangemaakt. Deze instelling voorkomt dat bij elke test tickets op de TOPdesk omgeving worden aangemaakt:

```
if not settings.TOPDESK_ENABLED:
    logger.warning("TOPdesk uitgeschakeld: mock ticket wordt opgeslagen")

    if writer is not None:
        writer(data)
    else:
        write_mock_ticket(data)

    return {"number": "MOCK-1234", "id": "mock_id"}

return create_topdesk_incident(data)
```

Figuur 30: Aanmaak mock ticket indien gewenst

Al deze informatie wordt vervolgens teruggekoppeld naar de gebruiker, zodat deze een bevestiging krijgt van de aangemaakte aanvraag.

3.5. Backend

De backend vormt de centrale laag waar alle componenten van de applicatie samenkomen. Waar in de voorgaande secties de afzonderlijke functionaliteiten zoals ingestie, RAG en intake werden uitgewerkt, ligt de focus hier op hoe deze onderdelen met elkaar verbonden worden en toegankelijk gemaakt worden via een API.

De backend is opgebouwd met FastAPI. Deze verwerkt inkomende requests, roept de juiste services aan en stuurt de resultaten terug naar de frontend in Streamlit.

API-laag en routing

De backend maakt gebruik van FastAPI om een REST API aan te bieden. Deze API laat toe om de verschillende functionaliteiten van de applicatie aan te roepen via HTTP-requests.

De applicatie is opgesplitst in meerdere routes, waarbij elke groep endpoints een specifieke verantwoordelijkheid heeft. Op deze manier blijft de structuur overzichtelijk en wordt de logica gescheiden per functionaliteit.

Onderstaande code toont hoe de verschillende routes worden geregistreerd in de applicatie:

```
app.include_router(intake_router)
app.include_router(rag_router)
app.include_router(admin_router)
app.include_router(feedback_router)
```

Figuur 31: Registratie van API-routes in FastAPI

De vier grote endpoints binnen de applicatie zijn:

- **Vraagverwerking (rag_router):** endpoint voor het stellen van gebruikersvragen, waarbij de RAG-pipeline wordt aangeroepen
- **Intake (intake_router):** endpoints voor het starten en verwerken van de intakeprocedure
- **Admin (admin_router):** endpoints voor het beheren van bronnen en het starten van ingestie
- **Feedback (feedback_router):** endpoint voor het opslaan van gebruikersfeedback (duimpjes)

Wanneer een request binnenkomt op een endpoint, wordt deze doorgestuurd naar de bijhorende service. De routes zelf bevatten zo weinig mogelijk logica en fungeren voornamelijk als tussenlaag tussen de API en de service-laag. Dit zorgt voor een duidelijke scheiding tussen de verwerking van HTTP-requests en de effectieve logica.

Daarna wordt gebruikgemaakt request-modellen om inkomende data te valideren. Deze modellen definiëren de structuur van de input en zorgen ervoor dat enkel geldige data verwerkt wordt door de backend.

Ter illustratie is hieronder een voorbeeld van een request-model voor de feedback:

```
class FeedbackRequest(BaseModel):
    """
    Model voor gebruikersfeedback op gegenereerde antwoorden.

    Attributes:
        query (str): De originele gebruikersvraag.
        answer (str): Het gegenereerde antwoord van de LLM.
        score (Literal["up", "down"]): Positieve ("up") of negatieve ("down") feedback.
    """

    query: str
    answer: str
    score: Literal["up", "down"]
```

Figuur 32: Request model voor feedback

In deze routers worden dus endpoints gedefinieerd waarin functies worden aangeroepen die zich in de services folder bevinden. Zo ziet bijvoorbeeld de "/ask"-endpoint in *rag_routes* eruit in de code:

```
@router.post("/ask")
def ask(payload: AskRequest) -> dict:
    """
    Verwerkt een gebruikersvraag via de RAG-pipeline.

    Args:
        payload (AskRequest): Bevat de vraag van de gebruiker.

    Returns:
        dict: Het gegenereerde antwoord met bijhorende bronnen en eventuele metadata.
    """
    return answer(payload.question)
```

Figuur 33: Endpoint voor vraagwerking

Dit is de functie die de RAG-pipeline uitvoert en vervolgens een antwoord genereert bij genoeg resultaten.

Voor het beheren van bronconfiguratie in de *admin_router* wordt gebruikgemaakt van een configuratie laag die eerder al werd toegelicht in de ingestie. De admin endpoints laten toe om deze configuratie dynamisch te laden en aan te passen, waarna de ingestie opnieuw kan worden uitgevoerd.

Service-laag en orchestratie

Achter de API-laag bevindt zich de service-laag. Deze services zorgen ervoor dat de effectieve logica wordt uitgevoerd. Elke service heeft een specifieke verantwoordelijkheid en vormt de brug tussen de API en de onderliggende componenten van de applicatie. De services roepen op hun beurt de eerder beschreven componenten aan, zoals de RAG-pipeline en de intakeflow.

De services binnen de applicatie zijn:

- **qa_service:** verwerkt gebruikersvragen en start de RAG-pipeline
- **intake_service:** beheert de intakeprocedure en de bijhorende interactie met de gebruiker
- **admin_service:** laat toe om ingestie op te starten en de bronconfiguratie te beheren
- **feedback_service:** verwerkt en bewaart feedback van gebruikers

Services voeren niet enkel individuele functies uit, maar coördineren ook de volledige flow. Zo zorgt bijvoorbeeld de *qa_service* ervoor dat de verschillende stappen binnen de RAG-pipeline zich in de juiste volgorde uitvoeren en dat het resultaat correct retourneert naar de API-laag.

Door deze duidelijke scheiding tussen routes en services blijft de applicatie modulair en onderhoudbaar. Nieuwe functionaliteiten kunnen eenvoudig toegevoegd worden door nieuwe services te definiëren of bestaande services uit te breiden, zonder impact op de API-laag.

Sessiebeheer

Binnen de backend wordt sessiebeheer toegepast om de staat van een intake over meerdere stappen heen te bewaren. Anders zou bij elke nieuwe request alle eerder ingevoerde informatie volledig verloren gaan. Daarom wordt bij het starten van een intake een unieke sessie-ID gegenereerd, die gebruikt wordt om alle antwoorden van een gebruiker te koppelen aan één sessie.

De sessies worden tijdelijk in-memory opgeslagen. Hierbij wordt per sessie-ID zowel de huidige stap in de intake als de verzamelde gegevens bijgehouden.

Onderstaande code toont de structuur van deze sessieopslag:

```
class SessionStore:
    def __init__(self) -> None:
        self.sessions: dict[str, dict[str, Any]] = {}

    def create(self, session_id: str) -> None:
        self.sessions[session_id] = {"step": 0, "data": {}}

    def get(self, session_id: str) -> Optional[dict[str, Any]]:
        return self.sessions.get(session_id)

    def update(self, session_id: str, key: str, value: Any) -> None:
        if session_id in self.sessions:
            self.sessions[session_id]["data"][key] = value

    def increment_step(self, session_id: str) -> None:
        if session_id in self.sessions:
            self.sessions[session_id]["step"] += 1

session_store = SessionStore()
```

Figuur 34: In-memory opslag van intake sessies

Logging & foutafhandeling

Om de onderhoudbaarheid van de applicatie te garanderen, is er binnen de backend een centraal logging- en foutafhandelingsysteem. Dit laat toe om fouten te detecteren, te analyseren en gepaste aanpassingen te maken zonder dat de applicatie onverwacht stopt zonder te weten waarom.

Logging gebeurt via een centrale configuratie die bij het opstarten van de applicatie wordt geïnitieerd. Hierbij wordt het logniveau bepaald op basis van een omgevingsvariabele, waardoor eenvoudig geschakeld kan worden tussen bijvoorbeeld ontwikkel- en productieomgevingen.

Onderstaande code toont de initialisatie van de logging configuratie:

```
def setup_logging() -> None:
    """
    Initialiseert de globale logging configuratie.

    Leest het log level uit de environment variabele LOG_LEVEL
    (default: INFO) en configureert het standaard logformaat.
    """
    level_name = os.getenv("LOG_LEVEL", "INFO").upper()
    level = getattr(logging, level_name, logging.INFO)

    logging.basicConfig(
        level=level,
        format="%asctime)s - %(name)s - %(levelname)s - %(message)s"
    )
```

Figuur 35: Initialisatie logging

De logging gebeurt via aparte logger-instanties per module. Dit maakt het mogelijk om berichten te categoriseren en gerichter te analyseren. In de code is een helperfunctie aanwezig om de logger op te halen waar toepasselijk.

Als voorbeeld is hieronder een screenshot van de terminal na een succesvolle retrieval:

```
2026-05-11 07:00:30,353 - rag.pipeline - INFO - Chunks geselecteerd door Reranker:
2026-05-11 07:00:30,353 - rag.pipeline - INFO - score 0.982 | Digitaal schriftelijk toet
2026-05-11 07:00:30,353 - rag.pipeline - INFO - score 0.802 | Handleiding voor het impor
2026-05-11 07:00:30,353 - rag.pipeline - INFO - score 0.794 | Proctorio
2026-05-11 07:00:30,353 - rag.pipeline - INFO - Relevantie gevonden! Best score: 0.9816
```

Figuur 36: Logging na retrieval

Naast logging wordt ook gebruikgemaakt van een eigen foutafhandeling via custom exceptions. Hiervoor werd een hiërarchie van exception classes gedefinieerd, waarbij een algemene basisfout zich uitbreidt naar meer specifieke fouttypes.

Door gebruik te maken van specifieke exception-types zoals fouten bij ingestie, validatie of externe services, kan gerichter gereageerd worden op verschillende foutscenario's. Zo is er ook duidelijke feedback aanwezig voor de ontwikkelaar aanwezig en verspil je geen tijd in het zoeken naar de reden van het foutlopen van de applicatie.

Onderstaande code toont deze structuur:

```
class AppError(Exception):
    pass

class IngestionError(AppError):
    pass

class SourceConfigError(AppError):
    pass

class ExternalServiceError(AppError):
    pass

class AppValidationError(AppError):
    pass
```

Figuur 37: Custom exceptions

Daarnaast worden deze custom exceptions in de backend gekoppeld aan specifieke HTTP-responses via exception handlers in de FastAPI applicatie (main.py). Voor elk type fout wordt een gepaste statuscode en foutboodschap teruggestuurd naar de applicatie.

Zo zijn validatiefouten bijvoorbeeld afgehandeld met een 400-statuscode, terwijl fouten bij externe services resulteren in een 502-statuscode. Op deze manier verkrijg je een duidelijke foutcommunicatie in de frontend.

Onderstaande code toont hoe een exception handler voor ingestie gedefinieerd is:

```
@app.exception_handler(IngestionError)
async def ingestion_exception_handler(request: Request, exc: IngestionError)
    """
    Verwerkt fouten tijdens de ingestie.
    """
    return JSONResponse(
        status_code=500, content={"detail": f"Ingestie fout: {str(exc)}"}
    )
```

Figuur 38: Exception handler voor ingestie

3.6. Frontend

De frontend vormt de gebruikersinterface waarmee eindgebruikers communiceren met het systeem. Via deze interface kunnen gebruikers vragen stellen, antwoorden ontvangen en indien nodig een intakeprocedure doorlopen. Daarnaast heeft een administrator via een aparte admin-tab de mogelijkheid om de ingestie uit te voeren en de bronconfiguratie te beheren.

De frontend werd ontwikkeld met Streamlit, een framework dat toelaat om op een eenvoudige en snelle manier een webapplicatie op te bouwen. De frontend communiceert met de backend via HTTP-requests naar de verschillende FastAPI-endpoints.

Chatinterface

In de chatinterface kan een gebruiker een vraag stellen. Na het indienen van de vraag wordt een HTTP-request gestuurd naar het “/ask”-endpoint van de backend. De backend verwerkt vervolgens deze vraag via de RAG-pipeline.

Bij genoeg relevante context wordt het antwoord weergegeven in de interface. Naast het gegenereerde antwoord zijn ook de bijhorende bronnen aanwezig in het antwoord. Daarnaast heeft de gebruiker de mogelijkheid om feedback te geven op het gegenereerde antwoord aan de hand van duimpjes.

Onderstaande screenshot geeft weer hoe een antwoord in Streamlit eruitziet:

Stel een vraag

Hoe neem ik kennisclips op in Kaltura?

Vraag stellen

Antwoord:

Om een kennisclip op te nemen in Kaltura, volg je de onderstaande stappen:

1. Installeer de applicatie Kaltura Capture op je toestel door via de Kaltura Mediaspace of de navigatieknop 'Mijn video's' in Canvas naar de knop '+Nieuwe toevoegen' te gaan en daar te kiezen voor 'Kaltura Capture'.
2. Nadat u de applicatie hebt geïnstalleerd, kan je de recorder lanceren door opnieuw de eerste stap bovenaan uit te voeren of door de Kaltura Capture applicatie vanop je toestel te starten.
3. Een recorder verschijnt op je scherm met de volgende functieknoppen:
 - Een knop om de opname van je scherm te selecteren, eventueel te kiezen welk scherm je wil opnemen (indien je met meerdere schermen werkt) en of je het volledige scherm of een bepaalde zone wil opnemen.
 - Een knop om de opname van je webcam aan/uit te zetten en eventueel te bepalen welke webcam opgenomen moet worden (indien je met meerdere camera's werkt).
4. Nadat u de gewenste instellingen hebt ingesteld, kan je beginnen met het opnemen van de kennisclip.

Was dit antwoord nuttig?



Bronnen:

- Kaltura - Kennisclips met de Kaltura Capture desktop recorder:
<https://stagetm.sharepoint.com/sites/LMS/SitePages/Kaltura---Kennisclips-met-Kaltura-Capture.aspx>

Figuur 39: Antwoord in Streamlit met optionele feedback & bronnen

In sommige scenario's levert de retrieval niet voldoende context op en wordt de intake getriggerd. In dat geval zal de respons aangeven dat een intakeprocedure moet worden opgestart:

The screenshot shows a web interface for an 'IT Assistent'. At the top, the title 'IT Assistent' is displayed in a large, bold, dark blue font. Below the title, there is a section titled 'Stel een vraag' (Ask a question) in a smaller font. Underneath this, a light blue input box contains the text 'Is er een tool waarmee ik grote bestanden kan verzenden?' (Is there a tool with which I can send large files?). Below the input box is a button labeled 'Vraag stellen' (Ask question). Further down, the section 'Intakeprocedure' (Intake procedure) is shown. It contains a light blue informational box with the text: 'Ik kan met deze informatie geen volledig antwoord geven. Daarom start ik een intakeprocedure zodat er een ticket kan worden aangemaakt. Dit ticket zal vervolgens door de ICTS-dienst worden bekeken en behandeld.' (I cannot give a full answer with this information. Therefore, I start an intake procedure so that a ticket can be created. This ticket will then be reviewed and processed by the ICTS service). Below this box, the text 'Beschrijf heel beknopt het probleem:' (Describe the problem very briefly:) is followed by another light blue input box labeled 'Jouw antwoord' (Your answer). At the bottom of this section is a button labeled 'Volgende' (Next).

Figuur 40: Opstart intake na vraag met onvoldoende context aanwezig

Intakeflow

Wanneer er bij een vraag onvoldoende relevante context beschikbaar is, wordt een intakeprocedure opgestart via de “/intake/start”-endpoint. Elke vraag wordt weergegeven in de interface en de gebruiker kan hierop een antwoord geven. Na het indienen van een antwoord wordt dit doorgestuurd naar het “/intake/answer”-endpoint, samen met de unieke sessie-ID die de intake identificeert.

Onderstaande screenshot toont een voorbeeld van een intakevraag in de frontend:

The screenshot shows a form titled 'Geef context aan je probleem/aanvraag:' (Give context to your problem/request:). Below the title, there is a section labeled 'Jouw antwoord' (Your answer). Inside this section, a light blue input box contains the text 'Ik ben op zoek naar software die me toelaat om grote bestanden te kunnen verzenden.' (I am looking for software that allows me to send large files). Below the input box is a button labeled 'Volgende' (Next).

Figuur 41: Intake vraag

Na het beantwoorden van alle vragen wordt de verzamelde informatie samengevoegd tot één context. Op basis van deze informatie wordt vervolgens gecontroleerd of er reeds een gelijkaardig ticket bestaat.

Indien er geen gelijkaardig ticket bestaat, wordt een ticket op TOPdesk aangemaakt. De gebruiker krijgt een overzicht van de aanvraag met de bevestiging dat een ticket is aangemaakt:



The screenshot shows a confirmation screen with a green header bar at the top that says "Intake afgerond". Below this is a section titled "Samenvatting van je aanvraag". Under the title, there are three lines of text: "Probleem / aanvraag: Tool voor verzenden grote bestanden", "Context: Ik ben op zoek naar software die me toelaat om grote bestanden te kunnen verzenden.", and "Doel: Software om grote bestanden mee te kunnen verzenden". At the bottom of the section, there is a green box containing a green checkmark icon followed by the text "Je ticket werd succesvol aangemaakt.", the "Ticketnummer: M260511 00001", and "De ICTS-dienst zal dit verder behandelen."

Intake afgerond

Samenvatting van je aanvraag

Probleem / aanvraag: Tool voor verzenden grote bestanden

Context: Ik ben op zoek naar software die me toelaat om grote bestanden te kunnen verzenden.

Doel: Software om grote bestanden mee te kunnen verzenden

✓ Je ticket werd succesvol aangemaakt.

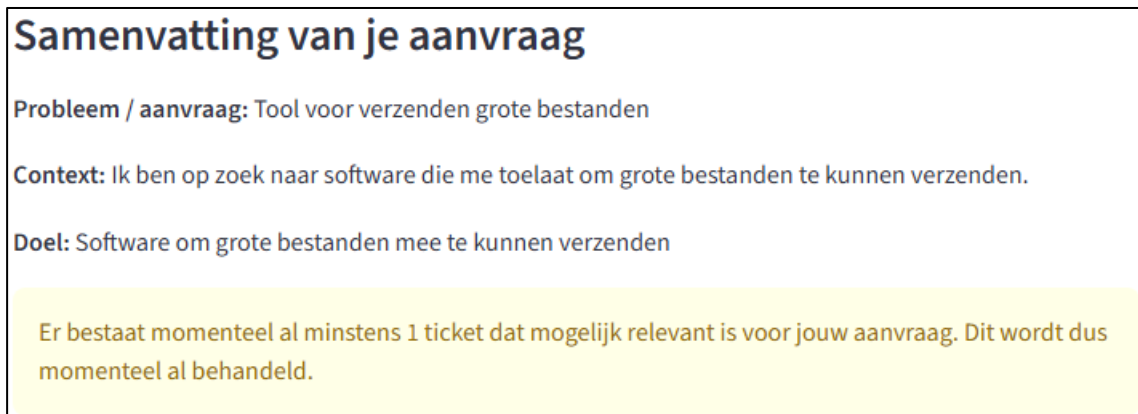
Ticketnummer: M260511 00001

De ICTS-dienst zal dit verder behandelen.

Figuur 42: Succesvolle ticketaanmaak

Wanneer een gelijkaardig ticket na de intake wel gedetecteerd wordt, zal de gebruiker geïnformeerd worden dat een gelijkaardig ticket reeds in behandeling is.

Onderstaande screenshot illustreert dit door het net aangemaakte ticket op te nemen in de ingestie en vervolgens opnieuw dezelfde aanvraag te doen:



The screenshot shows a confirmation screen with a yellow box at the bottom. The box contains the text "Er bestaat momenteel al minstens 1 ticket dat mogelijk relevant is voor jouw aanvraag. Dit wordt dus momenteel al behandeld." The rest of the screen is identical to Figure 42, showing the summary of the request.

Samenvatting van je aanvraag

Probleem / aanvraag: Tool voor verzenden grote bestanden

Context: Ik ben op zoek naar software die me toelaat om grote bestanden te kunnen verzenden.

Doel: Software om grote bestanden mee te kunnen verzenden

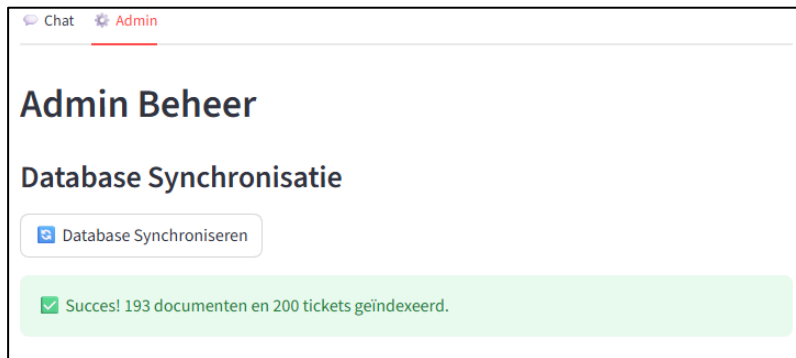
Er bestaat momenteel al minstens 1 ticket dat mogelijk relevant is voor jouw aanvraag. Dit wordt dus momenteel al behandeld.

Figuur 43: Detectie gelijkaardig ticket

Admininterface

Naast de chatinterface bevat de applicatie een admin-tab waarin beheerfunctionaliteiten voorzien zijn. In deze tab kan je de ingestie opstarten en de bronnen beheren.

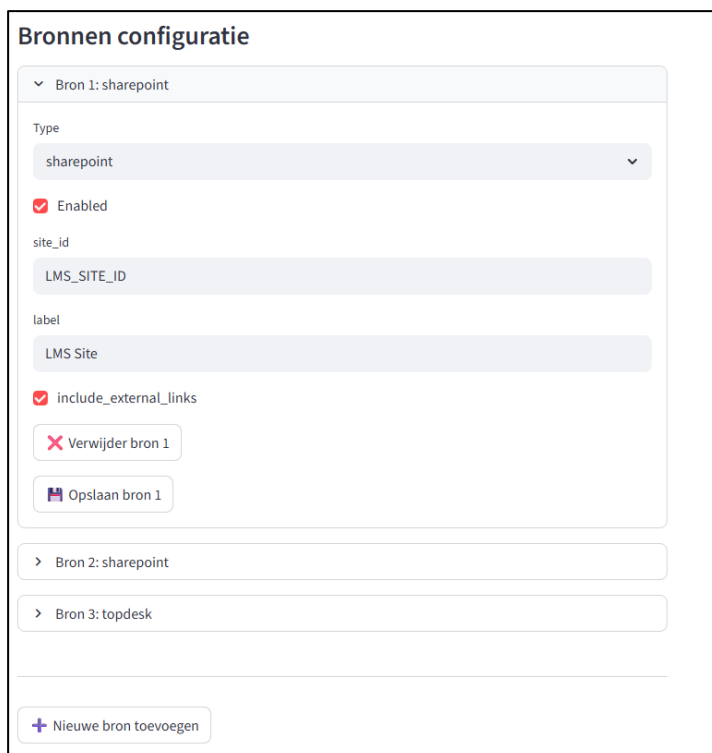
Een eerste belangrijke functionaliteit is het opstarten van de ingestie. Via een knop kan de administrator een synchronisatie van de vector database uitvoeren. Hierbij wordt een request gestuurd naar het “/ingest”-endpoint van de backend, waarna de ingestie wordt uitgevoerd. Bij een succesvolle ingestie ontvangt de gebruiker een succesmelding:



Figuur 44: Succesvolle ingestie

Daarnaast biedt de admininterface de mogelijkheid om de bronconfiguratie te beheren. De beschikbare bronnen worden opgehaald via de backend en weergegeven in de interface. Bij het opstarten en herladen van de Streamlit-applicatie wordt telkens een GET-request naar “/sources” uitgevoerd. Voor elke bron kunnen instellingen aangepast worden, zoals het type bron, configuratieparameters en activatiestatus.

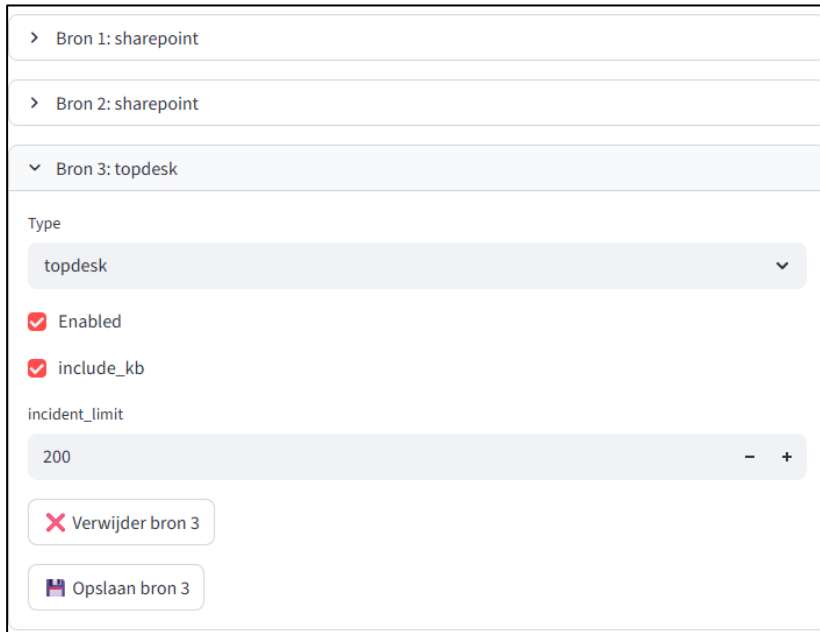
Onderstaande screenshot toont het overzicht van de bestaande bronnen:



Figuur 45: Overzicht bestaande bronnen

De configuratie van een bron is dynamisch opgebouwd op basis van een schema. Afhankelijk van het type bron worden verschillende invoervelden weergegeven.

Onderstaande screenshot toont het aanpassen van de bestaande TOPdesk-bron met een andere configuratieparameters dan SharePoint:

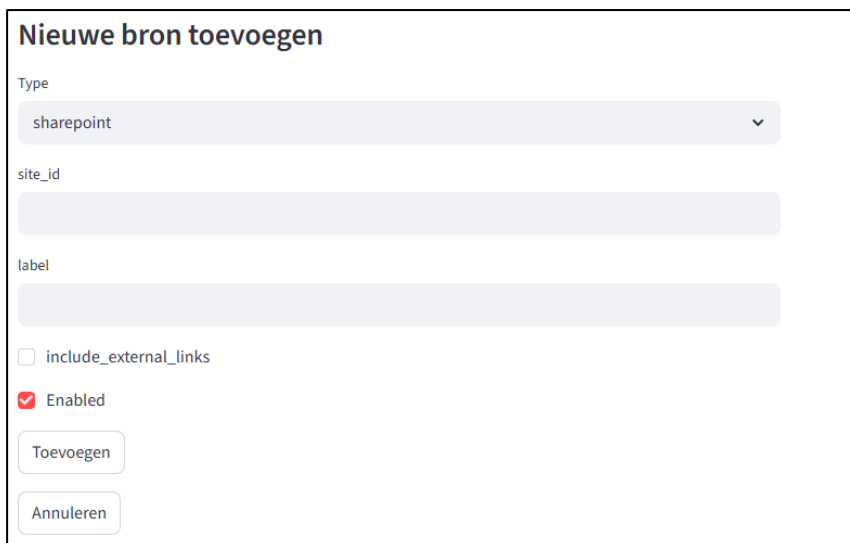


The screenshot shows a configuration panel for three data sources. The first two are 'Bron 1: sharepoint' and 'Bron 2: sharepoint'. The third, 'Bron 3: topdesk', is expanded. Within this section, the 'Type' dropdown is set to 'topdesk'. Below this, there are two checked checkboxes: 'Enabled' and 'include_kb'. The 'incident_limit' is set to 200, with minus and plus buttons for adjustment. At the bottom of the 'Bron 3' section are two buttons: 'Verwijder bron 3' (with a red X icon) and 'Opslaan bron 3' (with a floppy disk icon).

Figuur 46: Configuratie TOPdesk bron

Naast het aanpassen van bestaande bronnen is het ook mogelijk om nieuwe bronnen toe te voegen. Dit gebeurt opnieuw via het dynamisch schema gebaseerd op het gekozen type bron. De ingevoerde configuratie wordt vervolgens via een POST-request naar het “/sources”-endpoint doorgestuurd en opgeslagen in de *sources.json* file.

Onderstaande screenshot toont de aanmaak van een nieuwe bron. Het type bron bepaalt de configuratievelden:



The screenshot shows a form titled 'Nieuwe bron toevoegen'. The 'Type' dropdown is set to 'sharepoint'. Below it are two empty text input fields labeled 'site_id' and 'label'. There is an unchecked checkbox for 'include_external_links' and a checked checkbox for 'Enabled'. At the bottom are two buttons: 'Toevoegen' and 'Annuleren'.

Figuur 47: Toevoegen bron op basis van type

3.7. Evaluatie

Om de kwaliteit en betrouwbaarheid van het systeem te beoordelen, zijn er verschillende evaluatiemethoden aanwezig. Deze evaluatie combineert zowel handmatige testen als geautomatiseerde evaluatie en gebruikersfeedback.

Eenzijds zijn testsets gebruikt om de prestaties van verschillende LLM-modellen te vergelijken. Op deze manier kunnen ook verschillende parameters met elkaar vergeleken worden. Anderzijds is er een implementatie van een LLM-as-judge om gegenereerde antwoorden automatisch te evalueren op basis van meerdere criteria.

Ten slotte wordt ook gebruikersfeedback verzameld via de Streamlit-interface. Dit gebeurt aan de hand van duimpjes. Hierdoor ontstaat een realistisch beeld van de kwaliteit van de antwoorden die gebaseerd is op het perspectief van de gebruiker.

Door deze combinatie van evaluatiemethoden wordt zowel de technische correctheid alsook de bruikbaarheid van het systeem in kaart gebracht. Bovendien worden uiteindelijke aanpassingen steeds bepaald door de ontwikkelaar. AI wordt hierbij enkel gebruikt als ondersteunend hulpmiddel voor evaluatie, terwijl de ontwikkelaar de resultaten interpreteert en beslist of en welke aanpassingen nodig zijn.

Testsets en modelvergelijking

Voor de evaluatie van het systeem werd gebruikgemaakt van vooraf opgestelde testsets. Deze testsets bestaan uit een reeks representatieve gebruikersvragen, aangevuld met verwachte elementen in het antwoord. Ook wordt een score gegeven die tussen 0 en 2 ligt. Een 0 betekent een volledig fout antwoord, 1 een gedeeltelijk correct antwoord en 2 een volledig correct antwoord. Daarnaast is er een analysekolom voorzien waarin een uitgebreide analyse kan gegeven worden over het antwoord van de LLM. Zo kan bijvoorbeeld genoteerd worden of de opgehaalde chunks relevant waren en of de LLM hallucineerde. Dit laat toe om de kwaliteit van de gegenereerde antwoorden systematisch te beoordelen.

Met deze testset kunnen LLM-modellen met elkaar vergeleken worden. In dit project werd een vergelijking gemaakt tussen het Llama- en Mistral-model. Voor beide modellen werden identieke vragen gesteld, waarna de gegenereerde antwoorden vergeleken werden met de verwachte antwoorden.

Onderstaande screenshot toont een fragment van de gebruikte testset Mistral:

Vraag	Type	Verwacht resultaat	Score (0 Analyse
Wat is het verschil tussen Schoolyear en Proct	Normaal	Schoolyear = fraudepreventie, ve	2 Antwoord inhoudelijks correct en gebaseerd op juiste context. Retrieval en reranking goed. Anwrood bevat bijkomende informatie die overbodig kan beschouwd worden dus model kiest voor volledigheid over beknoptheid.
Ik wil examens thuis laten afleggen, welke to	Praktisch	Proctorio: opnames + detectie + g	2 Correct en beknopt antwoord met juiste bronselectie. Geen problemen in retrieval of reranking. Antwoord is direct en passend bij het vraagtype. Geeft misschien wat te veel info
Hoe activeer ik Proctorio in Canvas?	multi-step	via instellingen → navigatie → Se	1,5 Juiste informatie maar mist 1 stap omdat die uit chunk valt en niet in andere chunk aanwezig is bij top k

Figuur 48: Voorbeeld testset met vragen en analyse

Op basis van deze testsets konden verschillen in antwoordkwaliteit detected worden. Er werd gekeken naar de correctheid van de antwoorden, de volledigheid en de mate waarin het antwoord effectief inspeelt op de vraag van de gebruiker.

Het Llama-model behaalde een score van 20/44. Dezelfde tests uitvoeren met het zwaardere Mistral-model leverde een opmerkelijke verbetering naar 31,5/44. Het lichtere model heeft het moeilijk om nuance in vragen te begrijpen en ook om de beste informatie uit de meegeleverde chunks te vinden. Het is duidelijk dat dit voor het Mistral-model een stuk eenvoudiger is.

Een belangrijk probleem dat na deze tests naar voren kwam, was dat de retrieval op basis van de vraag af en toe niet de juiste chunks meegaf aan de LLM. Dit maakt het bijgevolg onmogelijk voor de LLM om een correct antwoord te genereren, ongeacht hoe capabel het model is. Een volgende stap was het aanpassen van parameters. Uiteindelijk is een test uitgevoerd met een chunk grootte van 512 (origineel 700) en een overlap van 200 (origineel 100). Na de volledige testset te doorlopen, verbeterde de score naar 38,5/44.

Een voorbeeld van een nog resterend probleem in de analyse is een vraag die bestaat uit twee elementen:

Ik wil groepswork evalueren en ook examens op afstand afnemen, welke tools gebruik ik best? meerdere br	Buddycheck	correct voor proctorio, maar mist buddycheck, heeft dus moeite met splitsen van vraag of mogelijk niet genoeg overlap tussen groepswork evalueren en peer assessment, Informatie over beide tools is niet aanwezig in dezelfde chunk en retriever heeft het dus moeilijk om voor beide tools 1 apart naar chunks te zoeken
---	------------	--

Figuur 49: Voorbeeld 1 van fout antwoord

De retrieval heeft het lastig met het herkennen van de twee gezochte elementen in de vraag en zal uiteindelijk enkel chunks meegeven over één onderwerp (hier Proctorio). De gewenste chunks die informatie moeten bevatten over het andere onderwerp (hier BuddyCheck) worden niet opgehaald.

Een ander voorbeeld is volgende vraag:

Ik wil kennisclips opnemen, kan dat met Proctorio?	trick	nee- Procto	Fout antwoord door verkeerde interpretatie van context. De vermelding van "kennisclips" wordt foutief geïnterpreteerd 0 als functionaliteit van Proctorio.
--	-------	-------------	--

Figuur 50: Voorbeeld 2 van fout antwoord

Wat deze vraag moeilijk maakt voor de LLM is dat kennisclips voorkomt in een chunk die gaat over Proctorio. Bijgevolg vermoedt de LLM dat dit een functionaliteit is van deze tool. Zowel Mistral als Llama botst op deze vraag.

Uit het gebruik van deze testsets blijkt dat de kwaliteit van de retrieval en de gegenereerde antwoorden snel kan worden geëvalueerd en gericht kan worden verbeterd zonder veel tijdverlies aan onnodige aanpassingen op plaatsen die geen impact zullen hebben op de kwaliteit.

LLM-as-judge

Naast de handmatige evaluatie met de testsets wordt ook gebruikgemaakt van een automatische evaluatiemethode via een LLM-as-judge. Hierbij is een tweede LLM ingezet om de kwaliteit van gegenereerde antwoorden te beoordelen.

Voor elke gebruikersvraag is er niet alleen generatie van een antwoord, maar ook een evaluatie uitgevoerd door de judge op basis van de volgende drie criteria:

- Faithfulness: betrouwbaarheid ten opzichte van de context
- Relevance: relevantie ten opzichte van de vraag
- Usefulness: bruikbaarheid voor de gebruiker

Onderstaande code toont een deel van de prompt die gebruikt wordt voor deze beoordeling:

```
Beoordeel het antwoord op basis van de volgende criteria:
```

1. FAITHFULNESS (t.o.v. de context):
 - 0 = bevat duidelijke hallucinaties
 - 1 = grotendeels correct maar kleine afwijkingen
 - 2 = volledig gebaseerd op context
2. RELEVANCE (t.o.v. de vraag):
 - 0 = antwoordt niet op de vraag
 - 1 = gedeeltelijk irrelevant
 - 2 = volledig relevant
3. USEFULNESS (voor de gebruiker):
 - 0 = niet bruikbaar
 - 1 = deels bruikbaar
 - 2 = duidelijk en bruikbaar

Figuur 51: Gedeeltelijke prompt voor LLM-as-judge

Via de system prompt krijgt de LLM-as-judge richtlijnen over wat deze categorieën betekenen en hoe de score tussen 0 en 2 bepaald moet worden. Daarbovenop wordt de judge ook verplicht om zijn output in JSON-formaat te genereren. De judge ontvangt tijdens de evaluatie de originele vraag, de gebruikte context en het gegenereerde antwoord als input voor de beoordeling.

Tijdens runtime gebeurt deze evaluatie automatisch en dit verloopt asynchroon. Dit betekent dat de evaluatie geen impact heeft op de responstijd van de gebruiker. In de backend wordt deze evaluatie gestart in een aparte thread na het genereren van het antwoord.

Nadat de evaluatie is uitgevoerd, kan in de terminal de volgende output worden weergegeven:

```
2026-05-11 09:43:08,555 - services.qa_service - INFO - JUDGE: {'faithfulness': 2, 'relevance': 2, 'usefulness': 2, 'uitleg': 'Het antwoord is volledig gebaseerd op de context en beantwoordt de vraag met een duidelijke en bruikbare manier om een kennisclip op te nemen in Kaltura.'}
```

Figuur 52: Terminal output LLM-as-judge

Door gebruik te maken van een LLM-as-judge kan inzicht verkregen worden in de kwaliteit van de gegenereerde antwoorden, zonder dat elke output manueel beoordeeld moet worden. Op deze manier kunnen hallucinaties snel gedetecteerd worden. Op basis van deze evaluatie kan de ontwikkelaar beslissen om aanpassingen te maken aan de prompt van de LLM die de antwoorden genereert. Dit kan bijvoorbeeld door extra richtlijnen toe te voegen of door gebruik te maken van few-shot prompting, waarbij voorbeelden worden gegeven van hoe een antwoord idealiter gestructureerd wordt of welke toon gehanteerd moet worden.

Onderstaande code toont een voorbeeld van few-shot prompting voor de LLM-as-judge prompt:

```
Geef ALLEEN JSON formaat zoals volgend voorbeeld:
```

```
{
  "faithfulness": 2,
  "relevance": 2,
  "usefulness": 1,
  "uitleg": "Het antwoord is volledig gebaseerd op de context en beantwoordt de vraag, maar mist enkele details die de bruikbaarheid verminderen."
}
```

Figuur 53: Few-shot prompting bij prompt voor judge

Gebruikersfeedback

Ten slotte wordt ook gebruikersfeedback verzameld via de frontend. Gebruikers hebben de mogelijkheid om feedback te geven op gegenereerde antwoorden aan de hand van duimpjes.

Na het ontvangen van een antwoord kan de gebruiker aangeven of dit antwoord nuttig was of niet. Deze feedback wordt via een HTTP-request ("/feedback") doorgestuurd naar de backend en daar verwerkt.

De feedback wordt opgeslagen in een JSONL-bestand, waarbij voor elke interactie de gebruikersvraag, het gegenereerde antwoord en de gegeven score worden bijgehouden. Daarnaast is ook een timestamp toegevoegd, zodat de feedback chronologisch geanalyseerd kan worden.

Onderstaande code toont hoe de feedback in het JSONL-bestand wordt opgeslagen:

```
{
  "timestamp": "2026-05-12T06:18:41.669860+00:00",
  "query": "Hoe neem ik kennisclips op in Kaltura?",
  "answer": " Om een kennisclip op te nemen in Kaltura,",
  "score": "up"
}
```

Figuur 54: Opgeslagen gebruikersfeedback in JSONL-bestand

Met de toevoeging van gebruikersfeedback kan een realistisch beeld gevormd worden van de kwaliteit van het systeem vanuit het perspectief van de gebruiker. Deze feedback vormt een waardevolle aanvulling op de andere evaluatiemethoden, aangezien ze gebaseerd is op echte gebruikerservaringen. Ook hier kan vervolgens beslist worden om gerichte aanpassingen te maken aan de prompt of om opnieuw gebruik te maken van few-shot prompting.

Door deze combinatie van evaluatiemethoden – testsets, een LLM-as-judge en gebruikersfeedback – verkrijg je een brede en complementaire evaluatie van het systeem. Waar testsets zorgen voor een gecontroleerde en objectieve vergelijking, en de LLM-as-judge automatische en schaalbare evaluatie mogelijk maakt, biedt gebruikersfeedback inzicht in de praktische bruikbaarheid vanuit het perspectief van de eindgebruiker.

Desondanks blijft de rol van de ontwikkelaar essentieel. De resultaten van deze evaluaties worden steeds geïnterpreteerd door de ontwikkelaar, die uiteindelijk beslist welke aanpassingen nodig zijn. Op deze manier blijft de mens centraal staan in het evaluatieproces, terwijl AI wordt ingezet als ondersteunend hulpmiddel.

3.8. Monitoring

Om inzicht te verkrijgen in de werking en prestaties van het systeem, is monitoring aanwezig via Langfuse. Deze tool maakt het mogelijk om de volledige RAG-pipeline te analyseren door middel van tracing en metrics.

Door de integratie met de backend worden alle stappen binnen de pipeline automatisch gelogd, waaronder retrieval, reranking, contextopbouw en antwoordgeneratie. Daarnaast biedt Langfuse inzicht in belangrijke metrics zoals latency en tokengebruik, die in een dashboard getoond worden.

Deze monitoring maakt het mogelijk om bottlenecks te identificeren en gerichte optimalisaties door te voeren, zowel op het niveau van de pipeline als van de LLM zelf.

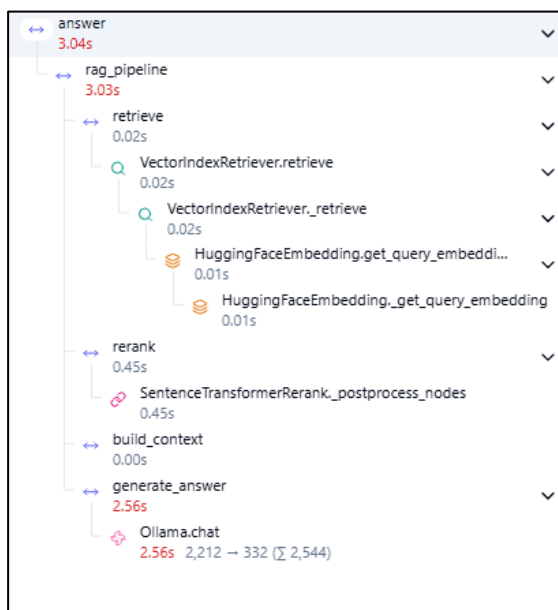
Tracing van de RAG-pipeline

Via tracing kan je de interne werking van de applicatie beter analyseren. Door middel van observaties kunnen de verschillende stappen binnen de RAG-pipeline afzonderlijk gemonitord worden. Binnen de code worden specifieke functies voorzien van een `@observe`-decorator, waardoor deze automatisch geregistreerd worden binnen Langfuse:

```
@observe(name="rag_pipeline")
def run_rag(
    query: str, collection: str = "docs", debug: bool = False
) -> Tuple[List[Any] | None, str | None, str | None]:
```

Figuur 55: Voorbeeld van observatie via `@observe` in de code

Onderstaande screenshot toont een voorbeeld van een volledige trace van de RAG-pipeline:



Figuur 56: Trace van volledige flow

Vervolgens kan op een individuele component (trace) worden geklikt om een overzicht te krijgen van de input, output, parameters, latency en andere relevante informatie. In de algemene flow is ook per onderdeel zichtbaar hoeveel latency elke stap inneemt. Merk op dat de waarden die in de screenshots met betrekking tot monitoring getoond worden, gebaseerd zijn op het Mistral-model dat uitgevoerd werd op een Deep Learning station.

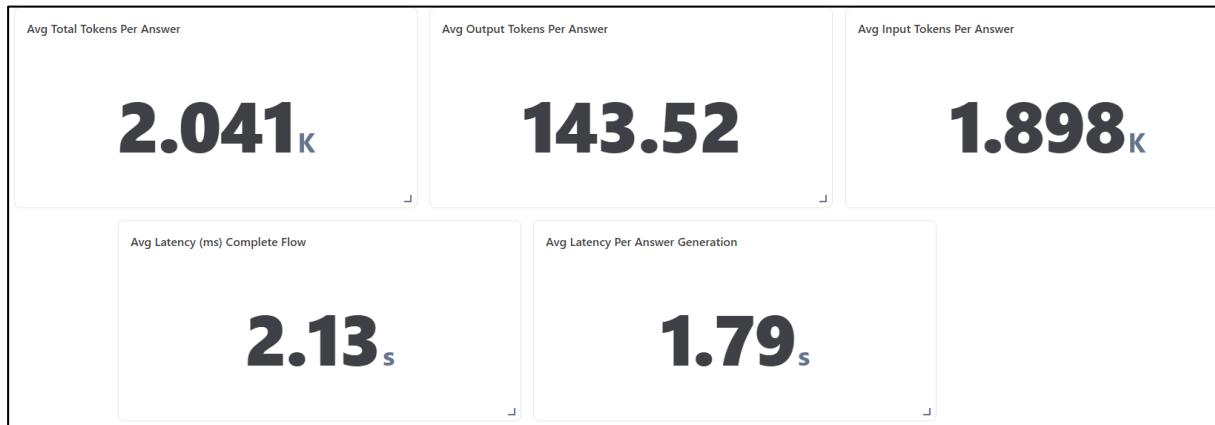
Naast de volgorde van de stappen biedt de trace ook inzicht in de latency van elke fase. Hieruit blijkt logischerwijs dat de antwoordgeneratie door de LLM het grootste aandeel heeft in de totale verwerkingstijd, terwijl stappen zoals retrieval en contextopbouw relatief weinig tijd in beslag nemen.

Door gebruik te maken van tracing kan snel geïdentificeerd worden waar eventuele vertragingen optreden binnen de pipeline. Dit maakt het mogelijk om gerichte optimalisaties door te voeren, bijvoorbeeld door het verbeteren van de retrievalstrategie of het aanpassen van modelparameters. Een bijkomend voordeel is dat de individuele stappen verder geanalyseerd kunnen worden. Zo kan bijvoorbeeld de functie `build_context` geopend worden om te bekijken hoe de aan de LLM meegegeven context er effectief uitziet in Markdown-structuur. Op die manier kan gecontroleerd worden of deze omzetting correct verloopt en of elementen zoals tabellen en lijsten correct en overzichtelijk zijn weergegeven.

Performance-analyse

Naast het analyseren van individuele traces biedt Langfuse ook de mogelijkheid om metrics te bekijken via een dashboard. Dit dashboard geeft een overzicht van belangrijke metrics zoals de gemiddelde latency van de volledige pipeline, de verwerking door de LLM en het tokengebruik.

Onderstaande screenshot toont een voorbeeld van dit dashboard:



Figuur 57: Dashboard latency en tokengebruik op Langfuse

Hier wordt duidelijk weergegeven dat het genereren van een antwoord duidelijk de meeste verwerkingstijd nodig heeft. Daarnaast wordt het tokengebruik ook weergegeven. Dit is een uiterst belangrijke metric, aangezien deze in een productieomgeving in grote mate de kost van het gebruik van een LLM bepaalt.

Door deze statistieken te analyseren, kan een inschatting gemaakt worden van zowel de kost van het aanroepen van de LLM als de latency tussen vraag en antwoord. Op basis hiervan kunnen gerichte aanpassingen in de code geëvalueerd worden, zoals het aanpassen van de contextgrootte om de responstijd te verbeteren. Op deze manier kan gezocht worden naar een evenwichtige balans tussen kwaliteit, snelheid en kost.

3.9. Kwaliteitswaarborging en onderhoudbaarheid

Naast de functionele implementatie is ook aandacht besteed aan de kwaliteit en onderhoudbaarheid van de code. Een goed werkende applicatie is immers niet voldoende.

Om dit te garanderen, werden verschillende technieken toegepast. Zo zijn unit tests en integratietests aanwezig om de correcte werking van de applicatie te verifiëren. Daarnaast is configuratie gecentraliseerd via omgevingsvariabelen, wat zorgt voor flexibiliteit en veiligheid bij het gebruik van externe diensten.

Verder werd de code voorzien van docstrings en type hints, waardoor de structuur en werking van de applicatie duidelijk gedocumenteerd zijn. Tot slot dragen linting en formatting bij aan een consistente codekwaliteit.

Unit testing

De correcte werking van de applicatie wordt verzekerd door middel van unit tests en integratietests. Deze tests zijn geïmplementeerd met behulp van *pytest* en maken het mogelijk om zowel individuele componenten als de interactie tussen verschillende onderdelen van de applicatie te controleren.

In deze tests zijn externe afhankelijkheden gemockt, waardoor verschillende scenario's gecontroleerd kunnen worden. Voorbeelden van deze scenario's zijn het ontbreken van een index, de connectie met SharePoint of de uitvoering van de pipeline.

Daarnaast zijn ook integratietests voorzien voor de API-laag. Met deze kan je controleren of fouten correct worden afgehandeld en of de juiste HTTP-statuscodes en foutmeldingen worden teruggegeven.

Onderstaande code toont een voorbeeld van een integratietest voor een validatiefout:

```
def test_validation_exception_handler(test_client):
    response = test_client.get("/test-validation-error")
    assert response.status_code == 400
    assert "Validatiefout" in response.json()["detail"]
```

Figuur 58: Integratietest voor validatiefout

Configuratie en omgevingsvariabelen

De configuratie van de applicatie is gecentraliseerd in een aparte settings-module, waarin alle belangrijke instellingen worden beheerd via omgevingsvariabelen. Hierdoor wordt een duidelijke scheiding gemaakt tussen configuratie en applicatielogica.

Onderstaande code toont een fragment van deze configuratie:

```
model_config = SettingsConfigDict(
    env_file=".env", env_file_encoding="utf-8", extra="ignore"
)

# SharePoint
SHAREPOINT_BASE_URL: Optional[str] = None
SHAREPOINT_CLIENT_ID: Optional[str] = None
SHAREPOINT_CLIENT_SECRET: Optional[str] = None
```

Figuur 59: Centrale configuratie via settings

Via deze aanpak kunnen gevoelige gegevens, zoals de API-sleutels en externe serviceconfiguraties, buiten de code worden gehouden. Dit verhoogt niet alleen de veiligheid, maar maakt het ook eenvoudiger om de applicatie in verschillende omgevingen te configureren.

Daarnaast zorgt deze structuur ervoor dat configuratiewijzigingen eenvoudig kunnen worden doorgevoerd zonder aanpassingen aan de code zelf.

Codekwaliteit en documentatie

De codebase is voorzien van docstrings en type hints, wat bijdraagt aan de leesbaarheid van de applicatie. Docstrings beschrijven de functionaliteit van functies, inclusief parameters en returnwaarden, terwijl type hints expliciet aangeven welke datatypes verwacht worden.

Onderstaande code toont een voorbeeld van een functie waarin deze principes worden toegepast:

```
@observe(name="answer")
def answer(query: str, debug: bool = True) -> dict[str, Any]:
    """
    Verwerkt een gebruikersvraag via de volledige RAG flow.

    Flow:
    - voert RAG pipeline uit
    - behandelt speciale gevallen (geen index / geen resultaten)
    - genereert antwoord en bronnen
    - start optioneel asynchrone evaluatie

    Args:
        query (str): De gebruikersvraag.
        debug (bool): Indien True, extra logging.

    Returns:
        dict[str, Any]: Antwoord en bijhorende bronnen.
    """
```

Figuur 60: Voorbeeld docstring en type hints bij answer() functie

Door het gebruik van deze technieken wordt de code beter begrijpbaar voor andere ontwikkelaars en wordt de kans op fouten verkleind.

4. Besluit

In dit project werd een AI-agent ontwikkeld op basis van Retrieval-Augmented Generation. Het doel was om gebruikersvragen op een efficiënte manier te beantwoorden door relevante context aanwezig in de kennisbronnen te combineren met generatieve AI, en waar nodig een intakeprocedure op te starten voor het aanmaken van een ticket op TOPdesk.

De aanleiding voor dit project lag in de bestaande werkwijze, waarbij gebruikers via de Service Catalog op SharePoint manueel door een verzameling van links moesten navigeren om informatie te vinden of een aanvraag te doen. Dit proces was tijdrovend en weinig gebruiksvriendelijk. Door de implementatie van een centrale chatinterface wordt deze interactie sterk vereenvoudigd. Gebruikers kunnen hun vraag rechtstreeks stellen, waarna het systeem automatisch de juiste informatie opzoekt of het verdere proces begeleidt. Dit resulteert in een meer efficiënte, gecentraliseerde en gebruiksvriendelijke aanpak.

Uit de implementatie blijkt dat de combinatie van retrieval, reranking en LLM-generatie een effectieve oplossing vormt. Hierbij speelt de kwaliteit van de retrieval een cruciale rol. Zonder relevante context is het voor de LLM onmogelijk om een correct antwoord te genereren. De toevoeging van een reranker bleek een duidelijke meerwaarde te zijn, aangezien deze zorgt voor een betere selectie van de relevante informatie op maat van de gebruikersvraag.

Een bijkomende uitdaging binnen dit project was het optimaliseren van de applicatie voor gebruik op een CPU-gebaseerde laptop. Deze beperking vereiste bewuste keuzes in modelgebruik en configuratie, maar leidde tegelijk tot een dieper inzicht in de werking en optimalisatie van RAG-systemen.

Evaluatie speelde een centrale rol binnen dit project. Door gebruik te maken van testsets, een LLM-as-judge en gebruikersfeedback kon de kwaliteit van het systeem vanuit verschillende perspectieven beoordeeld worden. Deze combinatie maakte het mogelijk om gerichte verbeteringen door te voeren en de werking van het systeem beter te begrijpen.

Ondanks de positieve resultaten zijn er ook enkele aandachtspunten. Het gebruik van LLM's brengt een inherent risico op hallucinaties met zich mee, waardoor kritische evaluatie van de gegenereerde antwoorden noodzakelijk blijft. Daarnaast heeft het tokengebruik een directe impact op de kost, wat een belangrijke factor vormt in een productieomgeving. Ook privacy en de afhankelijkheid van externe, vaak niet-Europese modellen zijn belangrijke aandachtspunten bij het verwerken van gevoelige informatie.

Met het oog op de toekomst zijn er verschillende uitbreidingsmogelijkheden. Zo kan onderzocht worden hoe meer autonome, agentic workflows geïntegreerd kunnen worden. Hierbij moet echter rekening gehouden worden met risico's zoals onnodige tool-calls en verhoogd tokengebruik als gevolg hiervan. Verder kan de retrieval geoptimaliseerd worden, bijvoorbeeld via betere chunking methodes of meer domeinspecifieke embeddings.

Tot slot kan geconcludeerd worden dat dit project aantoont dat RAG-systemen een krachtige en praktische oplossing vormen voor het verbeteren van kennisontsluiting en ondersteuning van gebruikers. Tegelijk benadrukt het project dat een doordachte implementatie, grondige evaluatie en continue monitoring essentieel zijn om tot betrouwbare en bruikbare resultaten te komen.

LITERATUURLIJST

- A., A. (2024, December 24). *Exploring “Langfuse” — An open-source LLM Engineering Platform*. Opgehaald van Medium: <https://ajay-arunachalam08.medium.com/exploring-langfuse-an-open-source-llm-engineering-platform-38cf5fe746e6>
- Arize Phoenix. (sd). *Open-source LLM tracing and evaluation* ___. Opgehaald van Arize Phoenix: <https://phoenix.arize.com/llamatrace/>
- Ashman, R. (2024, Februari 8). *The aRt of RAG Part 3: Reranking with Cross Encoders*. Opgehaald van Medium: <https://medium.com/p/688a16b64669>
- Awan, A. A. (2024, September 28). *Chroma DB Tutorial: A Step-By-Step Guide*. Opgehaald van Datacamp: <https://www.datacamp.com/tutorial/chromadb-tutorial-step-by-step-guide>
- Azevedo, P. (2025, September 3). *LangChain vs Llamaindex (2025) — Which One is Better?* Opgehaald van Medium: <https://medium.com/@pedroazevedo6/langgraph-vs-llamaindex-workflows-for-building-agents-the-final-no-bs-guide-2025-11445ef6fad6>
- Bansal, N. (2025, Juni 27). *Best Open-Source Embedding Models Benchmarked and Ranked*. Opgehaald van Supermemory: <https://supermemory.ai/blog/best-open-source-embedding-models-benchmarked-and-ranked/>
- BentoML. (sd). *LLM quantization*. Opgehaald van BentoML: <https://bentoml.com/llm/getting-started/llm-quantization>
- Bergmann, D. (sd). *What is instruction tuning?* Opgehaald van IBM: <https://www.ibm.com/think/topics/instruction-tuning#692473876>
- Brenndoerfer, M. (2025). *GPT-2: Scaling Language Models for Zero-Shot Learning*. Opgehaald van Michael Brenndoerfer: <https://mbrenndoerfer.com/writing/gpt-2-scaling-language-models-zero-shot-learning>
- C., E. (sd). *Ultimate Guide - The Best Open Source LLMs for RAG in 2026*. Opgehaald van SiliconFlow: <https://www.siliconflow.com/articles/en/best-open-source-LLMs-for-RAG>
- Dataloop. (2024). *Paraphrase Multilingual Mpnet Base V2*. Opgehaald van Dataloop: https://dataloop.ai/library/model/sentence-transformers_paraphrase-multilingual-mpnet-base-v2/
- Dataloop. (2025). *Bge M3*. Opgehaald van Dataloop: https://dataloop.ai/library/model/baai_bge-m3/
- Dataloop. (2025). *Gte Qwen2 1.5B Instruct*. Opgehaald van Dataloop: https://dataloop.ai/library/model/alibaba-nlp_gte-qwen2-15b-instruct/
- Docling. (sd). *LlamaIndex*. Opgehaald van Docling: <https://docling-project.github.io/docling/integrations/llamaindex>
- Gadesha, V. (sd). *What is prompt engineering?* Opgehaald van IBM: <https://www.ibm.com/think/topics/prompt-engineering>
- GigaSpaces. (2025, December 2). *The 6 Best Vector Database Solutions for RAG Applications*. Opgehaald van GigaSpaces: <https://www.gigaspace.com/blog/best-vector-database-solutions-for-rag-applications>
- Gondal, T. U. (2024, Mei 6). *LoRA: Low Rank Adaptation of Large Language Models*. Opgehaald van Medium: <https://medium.com/@tayyibgondal2003/loral-low-rank-adaptation-of-large-language-models-33f9d9d48984>
- Google. (2026, Januari 14). *Prompt engineering: overview and guide*. Opgehaald van Google Cloud: <https://cloud.google.com/discover/what-is-prompt-engineering>
- gpt2-large*. (sd). Opgehaald van Hugging Face: <https://huggingface.co/openai-community/gpt2-large>
- Hariharan. (2025, Juli 8). *Docling - Overview*. Opgehaald van Medium: <https://medium.com/@hari.haran849/docling-overview-b456139f3d04>
- Heidloff, N. (2025, September 25). *Introduction to Docling*. Opgehaald van Heidloff: <https://heidloff.net/article/docling/>
- Hugging Face. (sd). *Overview*. Opgehaald van Hugging Face: <https://huggingface.co/docs/transformers/v5.2.0/quantization/overview>
- Hugging Face. (sd). *Quantization*. Opgehaald van Hugging Face: https://huggingface.co/docs/transformers/main_classes/quantization
- K, J. (2024, September 29). *LLama 3.2 1B and 3B: small but mighty!* Opgehaald van Medium: <https://medium.com/pythoneers/llama-3-2-1b-and-3b-small-but-mighty-23648ca7a431>
- Krantz, T., & Jonker, A. (sd). *What is cosine similarity?* Opgehaald van IBM: <https://www.ibm.com/think/topics/cosine-similarity>

- Kumar, K. (2024, December 7). *The Illustrated Guide to Cross-Encoders: From Deep to Shallow*. Opgehaald van Medium: <https://medium.com/@kakumar1611/the-illustrated-guide-to-cross-encoders-from-deep-to-shallow-2a23a8630016>
- Kumar, R. (2025, Oktober 2). *A Deep Dive into Cross Encoders and How they work*. Opgehaald van Ranjan Kumar: <https://ranjankumar.in/a-deep-dive-into-cross-encoders-and-how-they-work>
- LangChain. (sd). *Build a RAG agent with LangChain*. Opgehaald van LangChain Docs: <https://docs.langchain.com/oss/python/langchain/rag>
- LangChain. (sd). *Retrieval*. Opgehaald van LangChain Docs: <https://docs.langchain.com/oss/python/langchain/retrieval>
- Llama3.2:1b*. (2025). Opgehaald van Ollama: <https://ollama.com/library/llama3.2:1b>
- Llama-3.2-1B*. (sd). Opgehaald van Hugging Face: <https://huggingface.co/meta-llama/Llama-3.2-1B>
- Llamahub. (sd). *Microsoft SharePoint Reader*. Opgehaald van Llamahub: <https://llamahub.ai/llreaders/llama-index-readers-microsoft-sharepoint?from=>
- Llamahub. (sd). *Microsoft SharePoint Reader*. Opgehaald van Llamahub: <https://llamahub.ai/llreaders/llama-index-readers-microsoft-sharepoint?from=all>
- LlamaIndex. (sd). *Docling Reader*. Opgehaald van LlamaIndex: https://developers.llamaindex.ai/python/examples/data_connectors/doclingreaderdemo/
- LlamaIndex. (sd). *Evaluating*. Opgehaald van LlamaIndex: <https://developers.llamaindex.ai/python/framework/understanding/evaluating/evaluating/>
- LlamaIndex. (sd). *Microsoft SharePoint Data Source*. Opgehaald van LlamaIndex: https://developers.llamaindex.ai/python/cloud/llamacloud/integrations/data_sources/sharepoint/
- LlamaIndex. (sd). *Welcome to LlamaIndex*. Opgehaald van LlamaIndex: <https://developers.llamaindex.ai/python/framework/>
- Local AI Master Research Team. (2026, Februari 4). *Chroma vs FAISS vs Qdrant vs Weaviate: Vector Database Comparison*. Opgehaald van Local AI Master: <https://localaimaster.com/blog/vector-databases-comparison>
- Ip, J. (2025, Oktober 10). *LLM-as-a-Judge Simply Explained: The Complete Guide to Run LLM Evals at Scale*. Opgehaald van Confident AI: <https://www.confident-ai.com/blog/why-llm-as-a-judge-is-the-best-llm-evaluation-method>
- Luna, I. L. (2025, Oktober 9). *LoRA Explained: Faster, More Efficient Fine-Tuning with Docker*. Opgehaald van Docker: <https://www.docker.com/blog/lora-explained/#:~:text=Instead%20of%20re%2Dtraining%20the,main%20model%20stays%20the%20same.&text=These%20few%20lines%20tell%20the,set%20of%20low%2Drank%20adapters.>
- Microsoft. (2025, Juni 9). *Register a client application in Microsoft Entra ID*. Opgehaald van Microsoft Learn: <https://learn.microsoft.com/en-us/azure/healthcare-apis/register-application>
- Microsoft. (sd). *RAG verhoogt de AI-nauwkeurigheid door externe kennis te integreren, zodat up-to-date, relevante antwoorden worden gegarandeerd*. Opgehaald van Microsoft: <https://azure.microsoft.com/nl-nl/resources/cloud-computing-dictionary/what-is-retrieval-augmented-generation-rag>
- Mitchell, T. (2024, December 20). *What are Embedding Models? An Overview*. Opgehaald van Couchbase: <https://www.couchbase.com/blog/embedding-models/>
- Models*. (sd). Opgehaald van Hugging Face: https://huggingface.co/models?pipeline_tag=text-generation
- Moez, A. (2025, Januari 18). *The Top 7 Vector Databases in 2026*. Opgehaald van Datacamp: https://www.datacamp.com/blog/the-top-5-vector-databases?utm_cid=19589720821&utm_aid=152984010854&utm_campaign=230119_1-ps-other~dsa-tofu~all_2-b2c_3-emea_4-prc_5-na_6-na_7-le_8-pdsh-go_9-nb-e_10-na_11-na&utm_loc=9197609-&utm_mtd=c&utm_kw=&utm_source=goo
- Mouschoutzi, M. (2025, September 24). *RAG Explained: Reranking for Better Answers*. Opgehaald van towards data science: <https://towardsdatascience.com/rag-explained-reranking-for-better-answers/>
- Murray, C. (2023, December 19). *Improving Llamaindex RAG performance with ranking*. Opgehaald van Medium: <https://colemurray.medium.com/enhancing-rag-with-baai-bge-reranker-a-comprehensive-guide-fe994ba9f82a>
- Noble, J. (sd). *What is LoRA?* Opgehaald van IBM: <https://www.ibm.com/think/topics/lora>
- Novogroder, I. (2025, April 3). *Top 9 RAG Tools to Boost Your LLM Workflows*. Opgehaald van lakeFS: <https://lakefs.io/blog/rag-tools/>

- Olteanu, A. (2024, September 26). *Llama 3.2 Guide: How It Works, Use Cases & More*. Opgehaald van Datacamp: <https://www.datacamp.com/blog/llama-3-2>
- Ong, R. (2024, Juli 3). *What Is Faiss (Facebook AI Similarity Search)?* Opgehaald van Datacamp: <https://www.datacamp.com/blog/faiss-facebook-ai-similarity-search>
- Pinecone. (sd). *Rerankers and Two-Stage Retrieval*. Opgehaald van Pinecone: <https://www.pinecone.io/learn/series/rag/rerankers/>
- Qwen. (2024). *LlamaIndex*. Opgehaald van Qwen: <https://qwen.readthedocs.io/en/latest/framework/LlamaIndex.html>
- Qwen2.5-1.5B-Instruct. (sd). Opgehaald van Hugging Face: <https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct>
- QwenTeam. (2024, September 18). *Qwen2.5: A Party of Foundation Models!* Opgehaald van Qwen: <https://qwen.ai/blog?id=qwen2.5>
- QwenTeam. (2024, September 18). *Qwen2.5-LLM: Extending the boundary of LLMs*. Opgehaald van Qwen: <https://qwen.ai/blog?id=qwen2.5-llm>
- Ragas. (2025, December 9). *Core Concepts*. Opgehaald van Ragas: <https://docs.ragas.io/en/stable/concepts/>
- Seahorse. (2025, Juli 1). *Parent-Child Chunking in LangChain for Advanced RAG*. Opgehaald van Medium: <https://medium.com/@seahorse.technologies.sl/parent-child-chunking-in-langchain-for-advanced-rag-e7c37171995a>
- Selbie, H., & Pakeman, T. (2024, December 19). *Optimizing RAG retrieval: Test, tune, succeed*. Opgehaald van Google Cloud: <https://cloud.google.com/blog/products/ai-machine-learning/optimizing-rag-retrieval>
- Shah, K. (2025, December 9). *5 Tools to Evaluate Prompt Retrieval Quality: The RAG Reliability Stack*. Opgehaald van Maxim: <https://www.getmaxim.ai/articles/5-tools-to-evaluate-prompt-retrieval-quality-the-rag-reliability-stack/>
- Shankar, D. (2025, November 13). *10 Best Open-Source LLM Models (2025 Updated): Llama 4, Qwen 3 and DeepSeek R1*. Opgehaald van Hugging Face: <https://huggingface.co/blog/daya-shankar/open-source-llms>
- Shastri, Y. (2025, Maart 5). *Top 10 RAG & LLM Evaluation Tools You Don't Want To Miss*. Opgehaald van Zilliz: <https://zilliz.com/learn/top-ten-rag-and-llm-evaluation-tools-you-dont-want-to-miss>
- Shin, M. (2025, Augustus 19). *10 Best RAG Tools and Platforms: Full Comparison [2026]*. Opgehaald van Meiliseach: <https://www.meiliseach.com/blog/rag-tools>
- Tahir, H. (2025, September 9). *LLamaIndex vs LangChain Which Framework Is Best for Agentic AI Workflows?* Opgehaald van ZenML: <https://www.zenml.io/blog/llamaindex-vs-langchain>
- Tuychiev, B. (2025, Oktober 15). *Docling: A Step-by-Step Guide to Building a Document Intelligence App*. Opgehaald van Datacamp: <https://www.datacamp.com/tutorial/docling>
- Varughese, J. (sd). *What is LangSmith?* Opgehaald van IBM: <https://www.ibm.com/think/topics/langsmith>
- What is a SharePoint team site?* (sd). Opgehaald van Microsoft: <https://support.microsoft.com/en-us/office/what-is-a-sharepoint-team-site-75545757-36c3-46a7-beed-0aaa74f0401e>
- Winland, V., & Russi, E. (sd). *What is LlamaIndex?* Opgehaald van IBM: <https://www.ibm.com/think/topics/llamaindex>
- Xu, S. (2025, Oktober 10). *The Best Open-Source Embedding Models in 2026*. Opgehaald van BentoML: <https://www.bentoml.com/blog/a-guide-to-open-source-embedding-models>
- Yarullin, R., & Parfenov, F. (2025, September 11). *Using LLMs as a Reranker for RAG: A Practical Guide*. Opgehaald van Fin: <https://fin.ai/research/using-llms-as-a-reranker-for-rag-a-practical-guide/>
- Yawen. (2024, December 26). *Dify v0.15.0: Introducing Parent-child Retrieval for Enhanced Knowledge*. Opgehaald van Dify: <https://dify.ai/blog/introducing-parent-child-retrieval-for-enhanced-knowledge>